

Sistema de Gestión de Oficina



Trabajo Final Integrador

Carrera: Tecnicatura universitaria en Programación

UTN

Integrantes:

- **Colombo Yago (Legajo: 14125) DNI:45292387**
- **Heinz Gaston (Legajo: 14133) DNI: 45191900**
- **Mattei Tomas (Legajo:14307) DNI: 42029438**

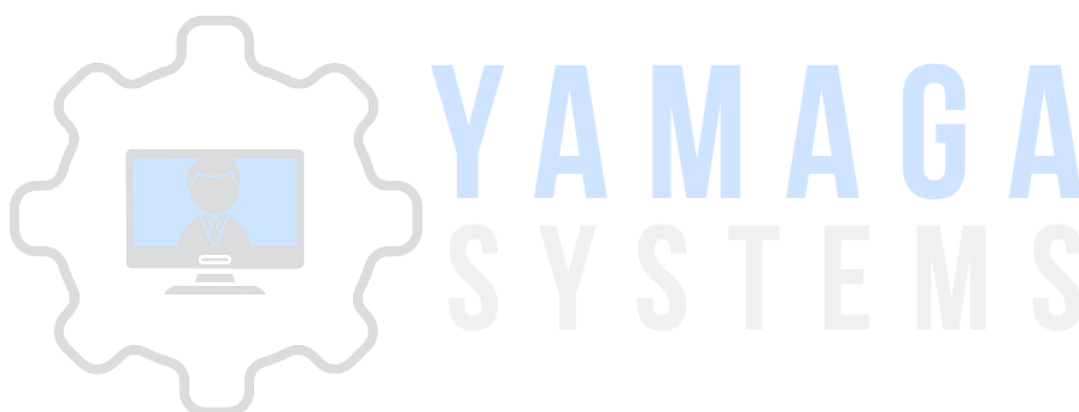
Fecha de presentación: 19/11/2025

Índices

Índices.....	2
Historial de Revisiones.....	5
Versión 0.1.0 - Inicio del Proyecto (Julio 2025).....	5
Versión 0.2.0 - Alpha (Agosto 2025).....	5
Versión 0.3.0 - Alpha (Agosto 2025).....	6
Versión 0.4.0 - Alpha (Agosto 2025).....	6
Versión 0.5.0 - Alpha (Septiembre 2025).....	6
Versión 0.6.0 - Alpha (Septiembre 2025).....	7
Versión 0.7.0 - Beta (Octubre 2025).....	7
Versión 0.8.0 - Beta (Octubre 2025).....	8
Versión 0.9.0 - Release Candidate (Noviembre 2025).....	8
Versión 1.0.0 - Versión Final (Noviembre 2025).....	9
Objetivo y descripción de la propuesta:.....	11
Objetivo General.....	11
Objetivos Específicos.....	11
Alcance del Sistema.....	11
Problema que Resuelve.....	12
Justificación Académica.....	12
Plan de trabajo.....	13
Fase 1 - Análisis y Diseño.....	13
Fase 2 - Implementación Core.....	13
Fase 3 - Módulos Principales.....	13
Fase 4 - Optimizaciones.....	13
Fase 5 - Deployment y Documentación.....	13
Tecnologías Utilizadas.....	14
Lenguajes de Programación.....	14
Frameworks.....	14
Sistema de Gestión de Base de Datos.....	14
Plataforma de Desarrollo.....	15
Plataforma de Producción.....	15
Control de Versiones.....	15
Herramientas de Desarrollo.....	15
Librerías y Paquetes Principales.....	16
Stack Completo.....	17
Arquitectura del Sistema.....	18
Patrón de Arquitectura: MVC + Repository.....	18
Estructura de Directorios.....	19
Notas sobre la estructura:.....	23
Flujo de una Operación Típica.....	23
Seguridad.....	24
Listado de Módulos Desarrollados.....	25
Módulo de Autenticación.....	25

Módulo de Gestión de Usuarios.....	25
Módulo de Gestión de Roles.....	26
Módulo de Gestión de Equipos.....	26
Módulo de Solicitudes de Préstamo (Trabajador).....	27
Módulo de Gestión de Solicitudes (Admin).....	28
Módulo Mis Equipos (Trabajador).....	28
Módulo de Mantenimiento.....	29
Módulo de Configuración del Sistema.....	30
Módulo de Auditoría.....	30
Módulo de Dashboards.....	31
Dashboard Admin.....	31
Dashboard Trabajador.....	31
Dashboard Mantenimiento.....	31
Módulo de Integraciones (Futuro).....	32
Gemini AI Service (Preparado pero no activo).....	32
Base de Datos - Esquema Completo.....	33
Diagrama Entidad-Relación (Descripción).....	33
Tablas Detalladas.....	34
Tabla: users.....	34
Tabla: roles.....	34
Tabla: equipment.....	35
Tabla: loans.....	35
Tabla: maintenance_requests.....	36
Tabla: audit_logs.....	37
Tabla: system_settings.....	37
Datos de Demostración.....	38
Manual de usuario.....	39
Introducción al Manual de Usuario.....	39
Inicio de Sesión.....	39
Manual del Administrador.....	40
Dashboard Principal del Administrador.....	40
Gestión de Equipos.....	41
Crear Nuevo Equipo.....	42
Editar Equipo Existente.....	43
Ver Historial de Préstamos.....	43
Gestión de Solicitudes de Préstamo.....	44
Aprobar Solicitud.....	45
Rechazar Solicitud.....	46
Ver Detalles de Solicitud.....	47
Gestion de Usuarios.....	47
Asignar Equipo Directamente a Usuario.....	48
Crear Nuevo Usuario.....	49
Editar Usuario.....	50
Gestión de Mantenimiento (Vista Admin).....	51

Configuración del Sistema.....	51
Manual del Trabajador.....	52
Dashboard del Trabajador.....	53
Solicitar un Préstamo.....	53
Mis solicitudes.....	54
Mis equipos.....	55
Reportar Problema.....	56
Manual del Personal de Mantenimiento.....	56
Dashboard de Mantenimiento.....	57
Gestión de Solicitudes.....	57
Completar Mantenimiento.....	58
Documentacion Oficial.....	60
Repositorios.....	60
Servicios Externos.....	60



Historial de Revisiones

Versión 0.1.0 - Inicio del Proyecto (Julio 2025)

Fecha: 15 de Julio de 2025

Estado: Planificación

Actividades:

- Definición de requisitos funcionales
- Diseño de arquitectura del sistema
- Modelado de base de datos (DER)
- Selección de tecnologías:
 - Backend: Laravel 12
 - Frontend: Filament 4.0
 - Base de datos: MySQL 8.0
 - Deploy: Railway
- Definición de roles y permisos
- Diseño de flujos de trabajo (préstamos, mantenimiento)
- Creación de repositorio Git

Versión 0.2.0 - Alpha (Agosto 2025)

Fecha: 1 de Agosto de 2025

Estado: Desarrollo

Configuración Inicial:

- Instalación de Laravel 12
- Instalación de Filament 4.0
- Configuración de AdminPanelProvider
- Autenticación con email y contraseña
- Panel de administración básico

Base de Datos:

- Migraciones base de Laravel
- Configuración de MySQL en local (Sail)
- Tabla roles con 3 roles predefinidos

Entorno de Desarrollo:

- Laravel Sail configurado
- Docker compose con MySQL, Redis, Mailpit
- Configuración de .env local

Versión 0.3.0 - Alpha (Agosto 2025)

Fecha: 10 de Agosto de 2025

Estado: Desarrollo

Nuevas Funcionalidades:

- Gestión de Equipos (EquipmentResource)
- CRUD completo con formularios Filament
- Campos: nombre, descripción, estado, imagen
- Filtros por estado en tabla
- Badges de color por estado

Migraciones:

- Tabla equipment con campos: name, description, status, image_path
- Enum para status: 'disponible', 'prestado', 'mantenimiento', 'baja'

Versión 0.4.0 - Alpha (Agosto 2025)

Fecha: 25 de Agosto de 2025

Estado: Desarrollo

Nuevas Funcionalidades:

- Gestión de Usuarios (UserResource)
- CRUD completo de usuarios
- Asignación de roles en formulario
- Contraseña opcional al editar (solo cambiar si se completa)
- Validación de email único

Seeders:

- RoleSeeder: 3 roles (Admin, Trabajador, Mantenimiento)
- UserSeeder: usuarios de prueba por rol
 - admin@gestionoficina.com
 - carlos@gestionoficina.com (trabajador)
 - pedro@gestionoficina.com (mantenimiento)
 - Contraseña: password123

Versión 0.5.0 - Alpha (Septiembre 2025)

Fecha: 5 de Septiembre de 2025

Estado: Testing funcional

Nuevas Funcionalidades:

- Solicitud de Préstamos (SolicitudPrestamoResource)
- Selección de equipo disponible en formulario
- Campo de motivo obligatorio

- Estado inicial "Pendiente"

Migraciones:

- Tabla loans con campos: equipment_id, user_id, requested_at, loan_date, return_date, status, approved_by, reason, rejection_reason, notes
- Enums para status: 'pendiente', 'rechazado', 'activo', 'devuelto'

Seeders:

- Equipos de prueba (10 equipos)
- Solicitudes de prueba en diferentes estados

Versión 0.6.0 - Alpha (Septiembre 2025)

Fecha: 20 de Septiembre de 2025

Estado: Testing funcional

Nuevas Funcionalidades:

- Módulo "Mis Solicitudes" (vista trabajador)
- Módulo "Mis Equipos" (equipos asignados al trabajador)
- Reportar problemas en equipos asignados
- Cancelar solicitudes pendientes
- Ver motivo de rechazo en solicitudes

Scopes en Models:

- Loan::pending(), Loan::active(), Loan::rejected(), Loan::returned()
- Equipment::available(), Equipment::loaned(), Equipment::maintenance(), Equipment::decommissioned()
- MaintenanceRequest::pending(), MaintenanceRequest::completed()
- Request::inProgress(), Request::completed()
- LoanPolicy: trabajador solo ve sus propias solicitudes
- EquipmentPolicy: trabajador no puede editar/eliminar equipos
- UserPolicy: solo admin puede gestionar usuarios

Políticas de Autorización:

Versión 0.7.0 - Beta (Octubre 2025)

Fecha: 10 de Octubre de 2025

Estado: Testing interno

Nuevas Funcionalidades:

- Gestión de Solicitudes (vista administrador)
- Aprobar solicitudes con fecha de devolución
- Rechazar solicitudes con motivo obligatorio
- Ver detalles completos de solicitudes
- Asignación directa de equipos (sin solicitud previa)

Automatizaciones:

- Al aprobar solicitud: equipo pasa a "Prestado", solicitud a "Activo"
- Al rechazar solicitud: equipo permanece "Disponible", solicitud a "Rechazado"
- Al reportar problema: equipo pasa a "Mantenimiento", préstamo activo se marca como devuelto

Validaciones:

- No permitir aprobar solicitudes si equipo no está disponible
- Fecha de devolución debe ser posterior a fecha de préstamo
- Motivo obligatorio al rechazar solicitud
- Descripción obligatoria al reportar problema

Versión 0.8.0 - Beta (Octubre 2025)

Fecha: 25 de Octubre de 2025

Estado: Testing interno

Nuevas Funcionalidades:

- Módulo de Mantenimiento completo
- Asignación de técnicos a solicitudes de mantenimiento
- Cambio de estado: Pendiente → En Proceso → Completado
- Dar de baja equipos irreparables
- Rechazar solicitudes de mantenimiento
- Historial de mantenimientos por equipo

Mejoras de UI:

- Badges de color por estado (equipos, solicitudes, mantenimiento)
- Iconos en menú lateral
- Tablas responsive
- Filtros colapsables en móvil

Migraciones:

- Tabla maintenance_requests con campos: equipment_id, reported_by, description, status, assigned_to, solution, result
- Índices en columnas de foreign keys

Versión 0.9.0 - Release Candidate (Noviembre 2025)

Fecha: 10 de Noviembre de 2025

Estado: Testing final

Cambios Implementados:

- Integración completa de auditoría en todas las operaciones
- Dashboards con gráficos CircleChart
- Widget de préstamos activos en dashboard trabajador
- Widget de solicitudes pendientes en dashboard mantenimiento
- Configuración del sistema (SystemSettings model + resource)

- Validación de límite de equipos por trabajador
- Alertas de vencimiento en dashboard y "Mis Equipos"

Correcciones:

- Filtros en tablas no persistían al navegar
- Badges de notificación no se actualizaban en tiempo real
- Fechas de devolución permitían valores pasados
- Al dar de baja un equipo, el préstamo activo no se cerraba automáticamente

Pruebas:

- Testing completo de flujos de préstamo
- Testing de mantenimiento (reparado vs baja)
- Testing de validaciones de fecha
- Testing de límites de equipos

Versión 1.0.0 - Versión Final (Noviembre 2025)

Fecha de Lanzamiento: 19 de Noviembre de 2025

Estado: Producción

Características Principales:

- Sistema completamente funcional en producción
- 11 módulos implementados y operativos
- 3 roles de usuario con permisos diferenciados
- Despliegue en Railway con base de datos MySQL 8.0
- Interfaz completa con Filament 4.0

Nuevas Funcionalidades:

- Dashboard personalizado por rol con widgets estadísticos
- Gráficos interactivos de distribución de equipos
- Sistema de badges para notificaciones en tiempo real
- Alertas de vencimiento configurables
- Auditoría completa de todas las operaciones
- Configuración de parámetros del sistema (max equipments_per_worker, dias_aviso_vencimiento)
- Historial completo de préstamos por equipo
- Historial completo de mantenimientos por equipo
- Filtros avanzados en todas las tablas
- Búsqueda en tiempo real

Módulos Implementados:

1. Gestión de Equipos (CRUD completo + historial)
2. Gestión de Usuarios (CRUD + asignación de roles)
3. Gestión de Roles (sistema de permisos)
4. Solicitudes de Préstamo (flujo completo: solicitar → aprobar/rechazar → devolver)
5. Mis Solicitudes (vista trabajador con filtros)
6. Mis Equipos (equipos asignados al trabajador logueado)
7. Gestión de Solicitudes (vista admin de todas las solicitudes)

8. Mantenimiento (flujo completo: reportar → asignar → reparar/dar de baja)
9. Configuración del Sistema (parámetros editables)
10. Auditoría (registro automático de operaciones)
11. Dashboards Personalizados (3 dashboards según rol)

Mejoras de Seguridad:

- Autenticación con Laravel Sanctum
- Validación de roles en todos los endpoints
- Políticas de autorización (EquipmentPolicy, LoanPolicy, etc.)
- Protección CSRF en formularios
- Sanitización de entradas
- Validación de fechas (DateValidationService)
- Límite de equipos por trabajador configurable

Optimizaciones de Rendimiento:

- Eager loading en relaciones Eloquent (->with())
- Índices en columnas de búsqueda frecuente
- Caché de consultas pesadas
- Paginación en todas las tablas (15 items por página)
- Queries optimizadas con scopes

Infraestructura:

- Despliegue automatizado en Railway
- Base de datos MySQL 8.0 en Railway
- HTTPS forzado en producción
- Manejo de proxies confiables (TrustProxies)
- Configuración de cookies seguras
- Variables de entorno protegidas

Testing:

- Tests unitarios para servicios (DateValidationService, LoanValidationService)
- Tests de feature para flujos completos
- Tests de políticas de autorización
- Cobertura de casos edge

Documentación:

- Manual de usuario completo (3 roles)
- Documentación técnica detallada
- Guía de despliegue en Railway
- Comandos de Sail documentados
- Diagramas de arquitectura
- Esquema de base de datos (DER)
- Preguntas frecuentes (FAQ)
- Glosario de términos

Objetivo y descripción de la propuesta:

Objetivo General

Desarrollar un sistema web para la gestión integral de equipos tecnológicos de oficina, que permita controlar préstamos, mantenimiento e inventario a través de un sistema de roles con permisos diferenciados.

Objetivos Específicos

1. Implementar un sistema de préstamos con flujo de aprobación
2. Controlar el inventario de equipos con estados y disponibilidad
3. Gestionar solicitudes de mantenimiento y reparaciones
4. Establecer un sistema de roles con permisos específicos
5. Mantener auditoría completa de todas las operaciones
6. Proporcionar dashboards personalizados según rol de usuario

Alcance del Sistema

el sistema **permite:**

- Solicitar préstamos de equipos (trabajadores)
- Aprobar/rechazar préstamos (administradores)
- Asignar equipos directamente sin solicitud previa (administradores)
- Reportar problemas en equipos (trabajadores)
- Gestionar mantenimiento y reparaciones (personal de mantenimiento)
- Dar de baja equipos irreparables
- Configurar parámetros del sistema (administradores)
- Auditar todas las operaciones realizadas

el sistema **NO permite:**

- Facturación o control de costos
- Gestión de proveedores
- Control de repuestos
- Sistema de notificaciones por email/SMS
- Reservas anticipadas de equipos

Problema que Resuelve

En muchas organizaciones, el control de equipos tecnológicos se realiza de forma manual (planillas Excel, correos, papeles), generando:

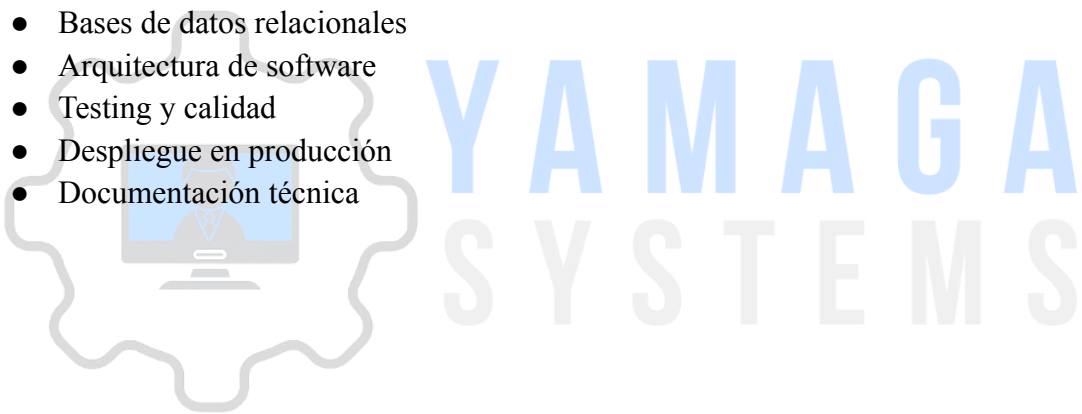
- Pérdida de equipos o extravío
- Falta de claridad sobre quién tiene qué equipo
- Demoras en aprobaciones de préstamos
- Equipos en mal estado sin reportar
- Imposibilidad de auditar operaciones

Este sistema centraliza toda la gestión en una plataforma web accesible desde cualquier lugar.

Justificación Académica

Este proyecto fue desarrollado como Trabajo Final Integrador, aplicando conocimientos de:

- Desarrollo web
- Bases de datos relacionales
- Arquitectura de software
- Testing y calidad
- Despliegue en producción
- Documentación técnica



Plan de trabajo

Fase 1 - Análisis y Diseño

- Relevamiento de requisitos
- Diseño de base de datos
- Definición de roles y permisos
- Diseño de arquitectura del sistema

Fase 2 - Implementación Core

- Configuración del entorno (Laravel Sail + Docker)
- Modelos y migraciones de base de datos
- Sistema de autenticación y roles
- CRUD básico de equipos y usuarios

Fase 3 - Módulos Principales

- Sistema de préstamos con aprobaciones
- Gestión de mantenimiento
- Dashboards personalizados por rol
- Widgets y estadísticas

Fase 4 - Optimizaciones

- Sistema de auditoría
- Optimizaciones de rendimiento
- Validaciones centralizadas
- Testing y corrección de bugs

Fase 5 - Deployment y Documentación

- Despliegue en Railway (producción)
- Documentación técnica
- Manual de usuario
- Datos de demostración

Tecnologías Utilizadas

Lenguajes de Programación

- **PHP 8.2:** Lenguaje principal del backend
- **JavaScript/Alpine.js:** Interactividad del frontend (incluido en laravel)
- **HTML5:** Estructura de vistas
- **CSS/Tailwind CSS:** Estilos y diseño responsive

Frameworks

- **Laravel 12:** Framework PHP principal
 - Versión más reciente con mejoras de rendimiento
 - Eloquent ORM para manejo de base de datos
 - Sistema de autenticación integrado
 - Middleware y gates para autorización
- **Filament 4.0:** Framework de administración
 - Interfaz de administración completa
 - CRUD generado automáticamente
 - Sistema de formularios y tablas
 - Widgets para dashboards
 - Sistema de notificaciones
- **Livewire 3:** Framework de componentes reactivos
 - Integrado con Filament
 - Interactividad sin escribir JavaScript
 - Actualizaciones en tiempo real

Sistema de Gestión de Base de Datos

- **MySQL 8.0:** Base de datos relacional
 - InnoDB como motor de almacenamiento
 - Transacciones ACID
 - Índices para optimización
 - Foreign keys para integridad referencial

Plataforma de Desarrollo

- **Docker:** Contenedorización
- **Laravel Sail:** Entorno de desarrollo dockerizado
 - PHP 8.4
 - MySQL 8.0
 - Redis (caché)
 - Mailpit (testing de emails)
 - Selenium (testing)
- **Composer:** Gestor de dependencias PHP
- **NPM:** Gestor de dependencias JavaScript
- **Vite:** Build tool para assets frontend

Plataforma de Producción

- **Railway:** Platform as a Service (PaaS)
 - Despliegue automático desde GitHub
 - Base de datos MySQL incluida
 - HTTPS automático
 - Monitoreo y logs
 - URL: <https://gestionoficina-production.up.railway.app>

Control de Versiones

- **Git:** Sistema de control de versiones
- **GitHub:** Repositorio remoto
 - Repositorio: <https://github.com/colomyago/gestionoficina>
 - Auto-deploy a Railway habilitado

Herramientas de Desarrollo

- **Visual Studio Code:** IDE principal
- **PHPUnit:** Testing unitario
- **Laravel Pint:** Code style (PSR-12)
- **Blade:** Motor de plantillas de Laravel

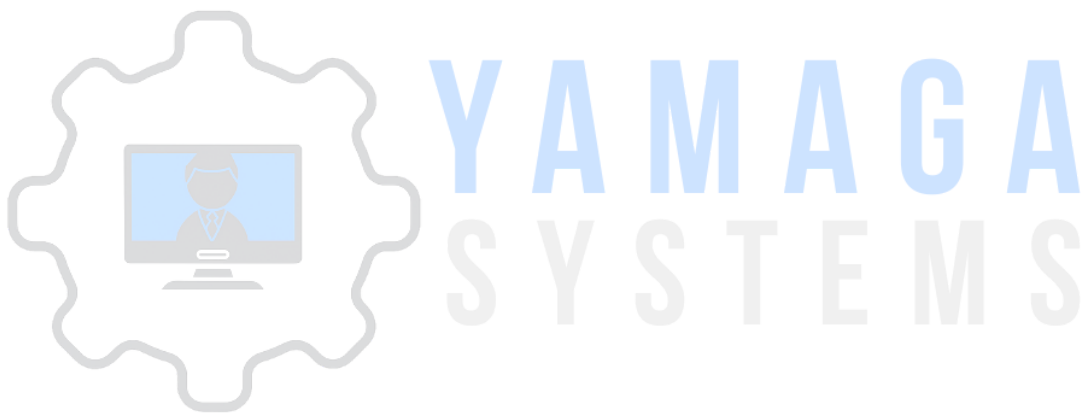
Librerías y Paquetes Principales

Backend (Composer):

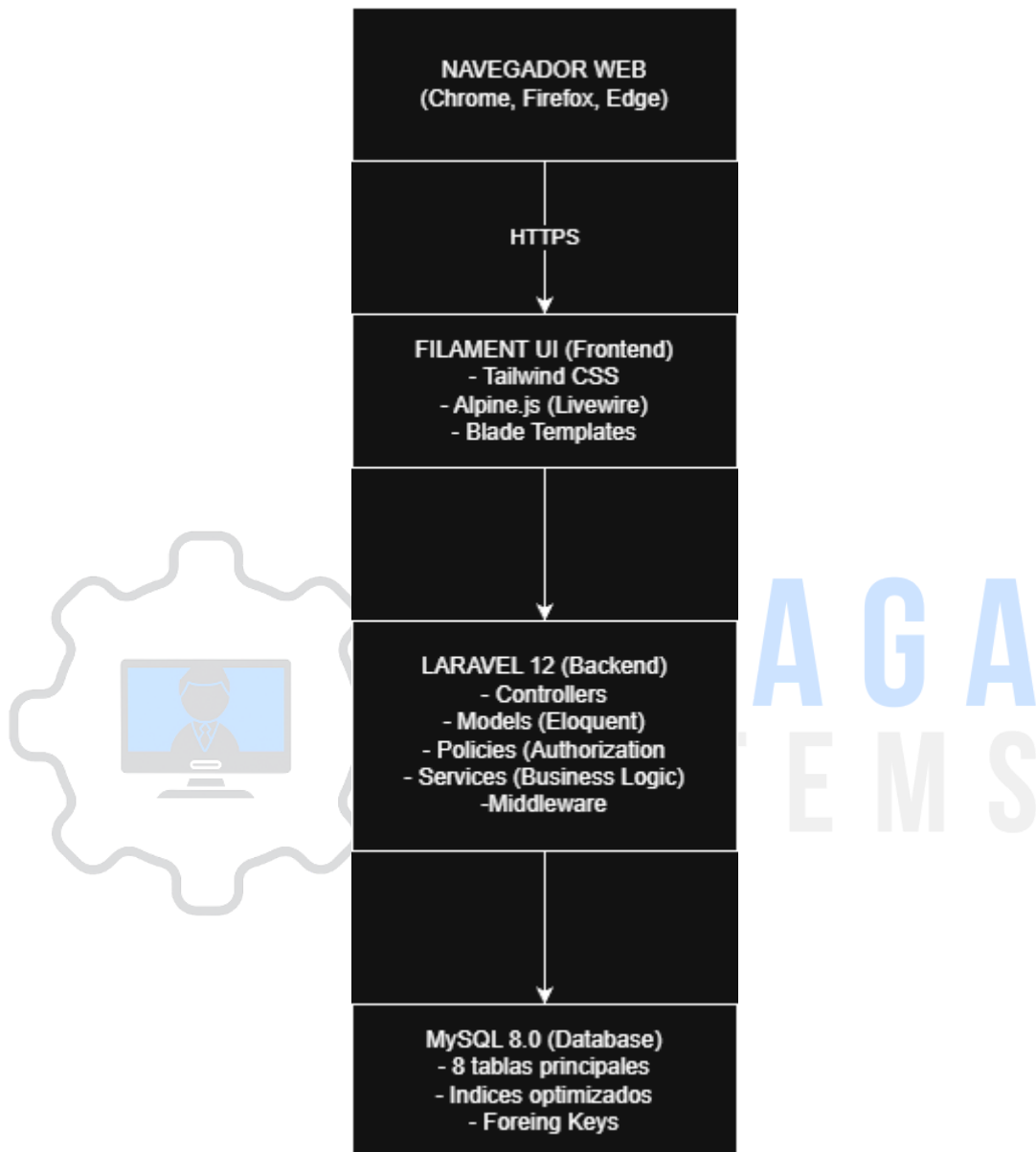
- laravel/framework: 12.0
- filament/filament: 4.0
- livewire/livewire: 3.0
- spatie/laravel-query-builder: "Para filtros avanzados"
- lab404/laravel-impersonate: "Impersonalizacion de usuarios"

Frontend (NPM):

- tailwindcss: 3.4
- alpinejs: 3.14
- autoprefixer: 10.4
- postcss: 8.4



Stack Completo



Arquitectura del Sistema

Patrón de Arquitectura: MVC + Repository

El sistema implementa el patrón Model-View-Controller (MVC) extendido con una capa de servicios:

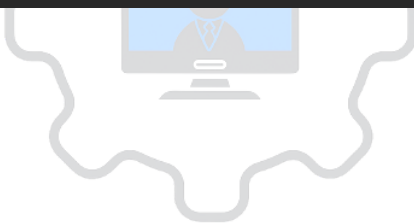


AMAGA
SYSTEMS

Estructura de Directorios

Directorio Raíz

gestionoficina/	
├─ app/	Código fuente de la aplicación
├─ bootstrap/	Archivos de arranque del framework
├─ config/	Archivos de configuración
├─ database/	Migraciones, seeders y factories
├─ public/	Punto de entrada web y assets públicos
├─ resources/	Vistas, CSS y JavaScript
├─ routes/	Definición de rutas
├─ storage/	Archivos generados, logs y caché
├─ tests/	Tests automatizados
├─ vendor/	Dependencias de Composer
├─ docs/	Documentación del proyecto
├─ .env	Variables de entorno
├─ artisan	CLI de Laravel
├─ composer.json	Dependencias PHP
├─ package.json	Dependencias JavaScript
├─ docker-compose.yml	Configuración de Docker
└─ Procfile	Configuración de Railway



SYSTEMS

app/ - Código de la Aplicación

```
app/
├── Filament/                                [Presentación]
│   ├── Resources/                          → CRUD (Equipos, Usuarios, Préstamos, etc.)
│   ├── Widgets/                           → Dashboards y estadísticas
│   └── Pages/                             → Páginas personalizadas
├── Models/                                [Datos]
│   ├── User.php, Role.php
│   ├── Equipment.php, Loan.php
│   ├── MaintenanceRequest.php
│   └── AuditLog.php, SystemSetting.php
├── Policies/                              [Autorización]
│   └── *Policy.php                        → Permisos por rol
├── Services/                              [Lógica de Negocio]
│   └── *ValidationService.php             → Reglas de validación
├── Http/                                  [HTTP]
│   ├── Controllers/
│   └── Middleware/
└── Providers/                             [Configuración]
    └── AppServiceProvider.php
```


database/ - Base de Datos

```
database/
├── migrations/                                Esquema de base de datos
│   ├── 0001_01_01_000000_create_users_table.php
│   ├── 2025_10_20_213921_create_roles_table.php
│   ├── 2025_09_17_061641_create_equipment_table.php
│   ├── 2025_10_20_000002_create_loans_table.php
│   ├── 2025_10_20_000003_create_maintenance_requests_table.php
│   ├── 2025_11_16_000001_create_system_settings_table.php
│   └── 2025_11_16_000003_create_audit_logs_table.php
├── seeders/                                  Datos de prueba
│   ├── DatabaseSeeder.php                    → Seeder principal
│   ├── RoleSeeder.php                       → 3 roles base
│   ├── UserSeeder.php                       → 15 usuarios de demo
│   └── EquipmentSeeder.php                  → 41 equipos de demo
└── factories/                                Factories para testing
    └── UserFactory.php
```

resources/ - Vistas y Assets

```
resources/
├── views/                                    Plantillas Blade
│   ├── components/                          → Componentes reutilizables
│   ├── layouts/                             → Layouts base
│   └── vendor/                              → Vistas de paquetes
├── css/
│   └── app.css                              → Estilos principales (Tailwind)
└── js/
    └── app.js                              → JavaScript principal
```

config/ - Configuraciones

config/	
├─ app.php	Configuración general
├─ auth.php	Autenticación
├─ database.php	Conexiones de BD
├─ filament.php	Configuración de Filament
├─ services.php	APIs externas
└─ session.php	Configuración de sesiones

tests/ - Testing

tests/	
├─ Feature/	Tests de funcionalidades
│ ├─ LoanTest.php	→ Tests de préstamos
│ ├─ EquipmentTest.php	→ Tests de equipos
│ └─ MaintenanceTest.php	→ Tests de mantenimiento
├─ Unit/	Tests unitarios
│ ├─ LoanValidationServiceTest.php	→ Tests de servicios
│ └─ DateValidationServiceTest.php	
└─ TestCase.php	Clase base de tests

docs/ - Documentación

docs/	
├─ DOCUMENTACION_TFI.md	Este documento
├─ SISTEMA_ROLES.md	Roles y permisos detallados
├─ FLUJO_COMPLETO_SISTEMA.md	Flujos de trabajo
├─ RAILWAY_DEPLOYMENT.md	Guía de deployment
├─ DATOS_DEMO.md	Usuarios y equipos de demo
├─ COMANDOS_SAIL.md	Referencia de comandos
└─ DIAGRAMA_BASE_DATOS.md	Diagrama ER

Raíz del proyecto/	
— .env	Variables de entorno
— .env.example	Plantilla de variables
— .gitignore	Archivos ignorados por Git
— artisan	CLI de Laravel
— composer.json	Dependencias PHP
— composer.lock	Versiones exactas de dependencias
— package.json	Dependencias JavaScript
— package-lock.json	Versiones exactas de npm
— vite.config.js	Configuración de Vite (build)
— tailwind.config.js	Configuración de Tailwind CSS
— phpunit.xml	Configuración de tests
— docker-compose.yml	Servicios Docker (Sail)
— Dockerfile	Imagen Docker personalizada
— Procfile	Comandos de Railway
— README.md	Información del proyecto

Notas sobre la estructura:

1. **Separación de responsabilidades:** Cada directorio tiene un propósito específico
2. **Escalabilidad:** Fácil agregar nuevos recursos siguiendo la estructura
3. **Mantenibilidad:** Código organizado por capas (Presentación, Lógica, Datos)
4. **Testing:** Tests organizados por tipo (Feature/Unit)
5. **Documentación:** Centralizada en carpeta docs/

Flujo de una Operación Típica

Ejemplo: Trabajador solicita un préstamo

1. **Presentación:** Usuario ingresa a SolicitudPrestamoResource y completa formulario
2. **Controller:** Filament Resource procesa la petición
3. **Policy:** LoanPolicy::create() verifica permisos del usuario
4. **Service:** LoanValidationService válida reglas de negocio:
 - Usuario no exceda límite de equipos (configurable)
 - Equipo esté disponible
 - Fechas sean válidas
5. **Model:** Se crea registro en tabla loans con estado pendiente
6. **Audit:** Se registra en audit_logs la acción
7. **View:** Usuario ve notificación de éxito y su solicitud aparece en la tabla

Ejemplo: Admin aprueba préstamo

1. **Presentación:** Admin ve solicitud en GestionSolicitudesResource
2. **Action:** Hace clic en "Aprobar" (table action)
3. **Policy:** LoanPolicy::approve() verifica que sea admin
4. **Transacción DB:**
 - Actualiza loans.status a activo
 - Actualiza equipment.status a prestado
 - Actualiza equipment.user_id con el trabajador
 - Registra fecha de préstamo
5. **Audit:** Registra aprobación en audit_logs
6. **Notificación:** Trabajador ve notificación (en sistema)

Seguridad

Autenticación:

- Laravel Sanctum para sesiones
- Contraseñas hasheadas con bcrypt
- Protección CSRF en todos los formularios

Autorización:

- Laravel Gates y Policies
- Verificación en cada operación CRUD
- Middleware para proteger rutas

Validación:

- Server-side validation en todos los formularios
- Validaciones de negocio en Services
- Sanitización de inputs

Base de Datos:

- Prepared statements (Eloquent)
- Transacciones para operaciones críticas
- Foreign keys con acciones en cascada

Producción:

- HTTPS forzado
- Cookies seguras
- Configuración de proxies confiables (Railway)
- Variables de entorno para credenciales

Listado de Módulos Desarrollados

Módulo de Autenticación

Descripción: Control de acceso al sistema

Funcionalidades:

- Login de usuarios
- Logout
- Recuperación de contraseña (estructura base)
- Sesiones persistentes

Archivos principales:

- config/auth.php
- app/Models/User.php
- Vistas de Filament (auto-generadas)

Módulo de Gestión de Usuarios

Descripción: CRUD completo de usuarios del sistema

Funcionalidades:

- Listar usuarios con filtros (por rol, búsqueda)
- Crear nuevos usuarios
- Editar información de usuarios
- Eliminar usuarios (soft delete opcional)
- Asignar roles
- Ver historial de préstamos por usuario
- Impersonar usuarios (para testing)

Roles que acceden:

- Admin (completo)

Archivos principales:

- app/Filament/Resources/Users/UserResource.php
- app/Models/User.php
- app/Policies/UserPolicy.php
- database/migrations/0001_01_01_000000_create_users_table.php

Tabla asociada: users

Módulo de Gestión de Roles

Descripción: Definición y asignación de roles

Funcionalidades:

- Listar roles existentes
- Ver usuarios por rol
- Crear/editar/eliminar roles (bloqueado en v1.0)

Roles predefinidos:

1. **Admin** (admin): Acceso total al sistema
2. **Trabajador** (trabajador): Solicita préstamos, reporta problemas
3. **Mantenimiento** (mantenimiento): Gestiona reparaciones

Roles que acceden:

- Admin (solo lectura)

Archivos principales:

- app/Filament/Resources/Roles/RoleResource.php
- app/Models/Role.php
- database/migrations/2025_10_20_213921_create_roles_table.php
- database/seeders/RoleSeeder.php

Tabla asociada: roles

Módulo de Gestión de Equipos

Descripción: Inventario completo de equipos tecnológicos

Funcionalidades:

- Listar equipos con filtros (estado, tipo, asignado a)
- Crear nuevos equipos
- Editar información de equipos
- Eliminar equipos (lógico)
- Cargar imagen del equipo
- Ver historial completo de préstamos
- Ver historial de mantenimientos
- Cambiar estado manualmente (disponible, prestado, mantenimiento, baja)
- Asignar directamente a un trabajador (sin solicitud previa)
- Liberar equipo (devolución directa)

Estados posibles:

- disponible: Listo para prestar
- prestado: Asignado a un trabajador

- mantenimiento: En reparación
- baja: Dado de baja permanentemente

Roles que acceden:

- Admin (completo)
- Trabajador (sólo lectura de equipos disponibles)
- Mantenimiento (solo lectura)

Archivos principales:

- app/Filament/Resources/Equipment/EquipmentResource.php
- app/Models/Equipment.php
- app/Policies/EquipmentPolicy.php
- database/migrations/2025_09_17_061641_create_equipment_table.php
- database/seeder/EquipmentSeeder.php (41 equipos de demo)

Tabla asociada: equipment

Módulo de Solicitudes de Préstamo (Trabajador)

Descripción: Trabajadores solicitan equipos

Funcionalidades:

- Ver MIS solicitudes (filtradas por usuario logueado)
- Crear nueva solicitud de préstamo
- Ver estado de mis solicitudes (pendiente/rechazado/activo/devuelto)
- Cancelar solicitud pendiente
- Ver motivo de rechazo (si aplica)
- Ver fecha estimada de devolución
- Validación: No exceder límite de equipos simultáneos
- Validación: No solicitar equipos ya prestados

Roles que acceden:

- Trabajador (sólo sus propias solicitudes)

Archivos principales:

- app/Filament/Resources/SolicitudPrestamoResource.php
- Usa modelo Loan.php

Tabla asociada: loans

Módulo de Gestión de Solicitudes (Admin)

Descripción: Administradores gestionan TODAS las solicitudes

Funcionalidades:

- Ver TODAS las solicitudes del sistema
- Filtrar por estado, usuario, equipo
- Aprobar solicitudes pendientes
- Rechazar solicitudes con motivo
- Establecer fecha estimada de devolución
- Agregar notas internas
- Ver historial completo de cada préstamo
- Forzar devolución de equipo
- Badge con contador de solicitudes pendientes

Acciones disponibles:

- **Aprobar:** Cambia estado a activo, equipo pasa a prestado
- **Rechazar:** Cambia estado a rechazado, equipo sigue disponible
- **Forzar devolución:** Devuelve equipo, cambia a devuelto

Roles que acceden:

- Admin

Archivos principales:

- app/Filament/Resources/GestionSolicitudesResource.php
- app/Services/LoanValidationService.php
- app/Policies/LoanPolicy.php

Tabla asociada: loans

Módulo Mis Equipos (Trabajador)

Descripción: Trabajadores ven equipos actualmente asignados

Funcionalidades:

- Ver equipos que ACTUALMENTE tengo prestados
- Ver fecha de préstamo y devolución estimada
- Reportar problema en equipo
- Alertas de vencimiento próximo (configurable)
- Widget con estadísticas personales

Acción destacada: Reportar Problema

1. Trabajador hace clic en "Reportar Problema"
2. Ingresa descripción del problema

3. Sistema crea MaintenanceRequest automáticamente
4. Equipo cambia a estado mantenimiento
5. Si había préstamo activo, se marca como devuelto automáticamente
6. Personal de mantenimiento recibe notificación (badge)

Roles que acceden:

- Trabajador (sólo sus equipos)

Archivos principales:

- app/Filament/Resources/MisEquiposResource.php
- app/Filament/Widgets/MyActiveLoansWidget.php
- app/Filament/Widgets/MisEquiposStatsWidget.php

Tabla asociada: loans (con filtro user_id)

Módulo de Mantenimiento

Descripción: Gestión de reparaciones y equipos en mal estado

Funcionalidades:

- Ver TODAS las solicitudes de mantenimiento
- Filtrar por estado (pendiente/en_proceso/completado/rechazado)
- Asignarse solicitudes de mantenimiento
- Cambiar estado a "En proceso"
- Completar reparación con solución
- Marcar equipo como "Reparado" (vuelve a disponible)
- Dar de baja equipo irreparable (estado baja)
- Rechazar solicitud de mantenimiento con motivo
- Ver historial de mantenimientos por equipo
- Badge con contador de solicitudes pendientes

Estados de mantenimiento:

- pendiente: Recién reportado
- en_proceso: Técnico trabajando en ello
- completado: Reparado o dado de baja
- rechazado: No requiere mantenimiento

Resultados posibles:

- reparado: Equipo vuelve a disponible
- dado_de_baja: Equipo pasa a baja permanentemente
- pendiente: Sin resolver aún

Roles que acceden:

- Mantenimiento (completo)
- Admin (completo)

Archivos principales:

- app/Filament/Resources/MantenimientoResource.php
- app/Models/MaintenanceRequest.php
- app/Policies/MaintenanceRequestPolicy.php
- app/Filament/Widgets/PendingMaintenanceWidget.php
- database/migrations/2025_10_20_000003_create_maintenance_requests_table.php

Tabla asociada: maintenance_requests

Módulo de Configuración del Sistema

Descripción: Parámetros configurables del sistema

Funcionalidades:

- Configurar límite de equipos por trabajador
- Configurar días de aviso antes de vencimiento
- Ver/editar configuraciones existentes
- Agregar nuevas configuraciones (manual en BD)

Configuraciones actuales:

- max equipments per worker: Máximo 5 equipos simultáneos (default)
- dias_aviso_vencimiento: Aviso 7 días antes (default)

Roles que acceden:

- Admin

Archivos principales:

- app/Filament/Resources/SystemSettingResource.php
- app/Models/SystemSetting.php
- database/migrations/2025_11_16_000001_create_system_settings_table.php

Tabla asociada: system_settings

Módulo de Auditoría

Descripción: Registro de todas las operaciones críticas

Funcionalidades:

- Registro automático de eventos
- Ver historial de auditoría (futuro dashboard)
- Rastrear quién hizo qué y cuándo
- Guardar valores anteriores y nuevos (JSON)
- Registrar IP y user agent

Eventos auditados:

- Aprobación de préstamos
- Rechazo de préstamos
- Devolución de equipos
- Creación de solicitudes de mantenimiento
- Reparaciones completadas
- Equipos dados de baja
- Asignación directa de equipos
- Cambios de estado

Archivos principales:

- app/Models/AuditLog.php
- database/migrations/2025_11_16_000003_create_audit_logs_table.php

Tabla asociada: audit_logs

Módulo de Dashboards

Dashboard Admin

Widgets:

- StatsOverviewWidget: Estadísticas generales (totales)
 - Total de equipos
 - Equipos prestados
 - Equipos disponibles
 - Solicitudes pendientes
 - En mantenimiento
- EquipmentChartWidget: Gráfico de equipos por estado
- RecentLoansWidget: Últimos 5 préstamos aprobados

Dashboard Trabajador

Widgets:

- MyActiveLoansWidget: Mis préstamos activos con alertas
- MisEquiposStatsWidget: Mis estadísticas personales
 - Equipos actualmente prestados
 - Total de préstamos históricos
 - Solicitudes pendientes

Dashboard Mantenimiento

Widgets:

- PendingMaintenanceWidget: Solicitudes de mantenimiento pendientes

Archivos principales:

- app/Filament/Widgets/ (todos los widgets listados)

Módulo de Integraciones (Futuro)

Gemini AI Service (Preparado pero no activo)

Descripción: Servicio para sugerencias inteligentes

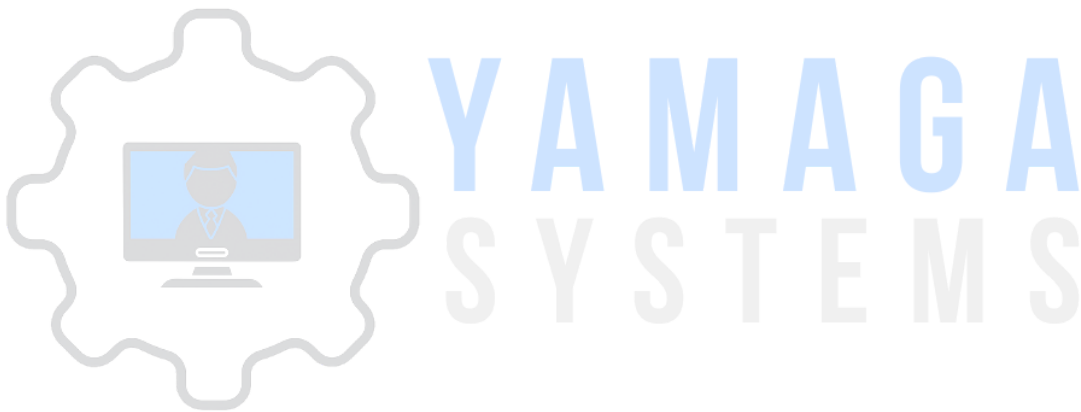
Funcionalidades preparadas:

- Análisis de patrones de préstamos
- Sugerencias de equipos según perfil de usuario
- Predicción de necesidades de mantenimiento

Archivo:

- app/Services/GeminiService.php

Estado:Preparado pero no implementado en UI

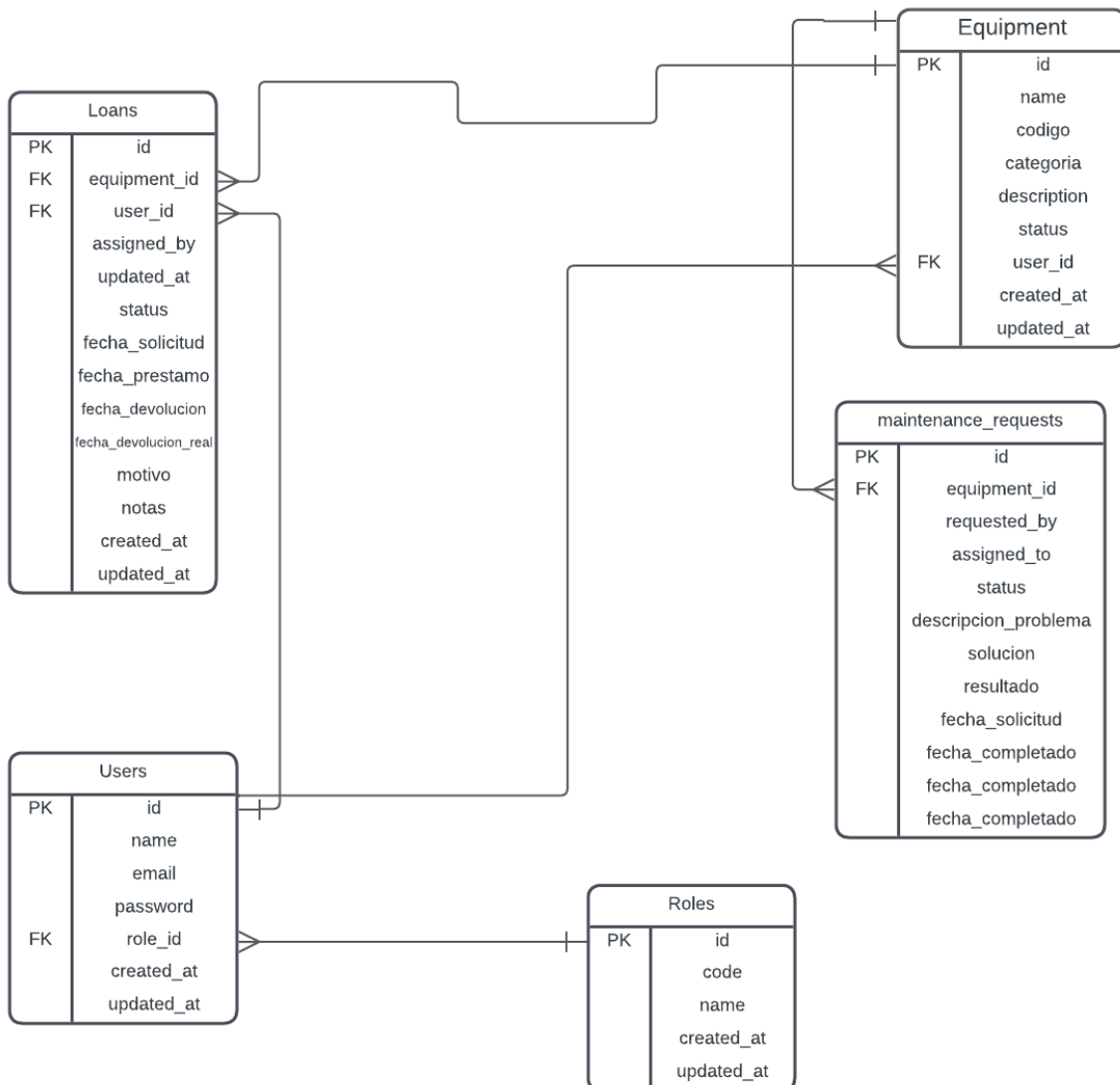


Base de Datos - Esquema Completo

Diagrama Entidad-Relación (Descripción)

Entidades principales:

1. users - Usuarios del sistema
2. roles - Roles de usuario
3. equipment - Equipos tecnológicos
4. loans - Préstamos de equipos
5. maintenance_requests - Solicitudes de mantenimiento
6. audit_logs - Registro de auditoría
7. system_settings - Configuración del sistema
8. sessions - Sesiones de usuario (Laravel)



Tablas Detalladas

Tabla: users

```
CREATE TABLE users (  
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  email_verified_at TIMESTAMP NULL,  
  password VARCHAR(255) NOT NULL,  
  role_id BIGINT UNSIGNED NULL,  
  remember_token VARCHAR(100) NULL,  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL,  
  
  FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE SET NULL,  
  INDEX idx_users_role_id (role_id),  
  INDEX idx_users_email (email)  
);
```

Tabla: roles

```
CREATE TABLE roles (  
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  code VARCHAR(255) UNIQUE NOT NULL, -- 'admin', 'trabajador', 'mantenimiento'  
  name VARCHAR(255) NOT NULL,      -- 'Administrador', 'Trabajador', 'Mantenimiento'  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL,  
  
  INDEX idx_roles_code (code)  
);
```

Tabla: equipment

```
CREATE TABLE equipment (  
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL,  
  description TEXT NULL,  
  status ENUM('disponible', 'prestado', 'mantenimiento', 'baja') DEFAULT 'disponible',  
  user_id BIGINT UNSIGNED NULL,      -- Usuario que actualmente lo tiene  
  image VARCHAR(255) NULL,          -- Ruta de imagen  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL,  
  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,  
  INDEX idx_equipment_status (status),  
  INDEX idx_equipment_user_id (user_id)  
);
```

Tabla: loans

```
CREATE TABLE loans (  
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  equipment_id BIGINT UNSIGNED NOT NULL,  
  user_id BIGINT UNSIGNED NOT NULL,      -- Quien solicita/tiene  
  assigned_by BIGINT UNSIGNED NULL,      -- Admin que aprobó  
  status ENUM('pendiente', 'rechazado', 'activo', 'devuelto') DEFAULT 'pendiente',  
  fecha_solicitud DATE NULL,  
  fecha_prestamo DATETIME NULL,          -- Cuando se aprobó y entregó  
  fecha_devolucion DATE NULL,            -- Fecha estimada de devolución  
  fecha_devolucion_real DATETIME NULL,    -- Cuando se devolvió realmente  
  motivo TEXT NULL,                     -- Por qué lo necesita  
  notas TEXT NULL,                       -- Notas del admin  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL,  
  
  FOREIGN KEY (equipment_id) REFERENCES equipment(id) ON DELETE CASCADE,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (assigned_by) REFERENCES users(id) ON DELETE SET NULL,  
  INDEX idx_loans_status (status),  
  INDEX idx_loans_user_id (user_id),  
  INDEX idx_loans_equipment_id (equipment_id),  
  INDEX idx_loans_fecha_devolucion (fecha_devolucion)  
);
```

Tabla: maintenance_requests

```
CREATE TABLE maintenance_requests (  
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  equipment_id BIGINT UNSIGNED NOT NULL,  
  requested_by BIGINT UNSIGNED NOT NULL,    -- Quien reportó  
  assigned_to BIGINT UNSIGNED NULL,        -- Técnico asignado  
  status ENUM('pendiente', 'en_proceso', 'completado', 'rechazado') DEFAULT 'pendiente',  
  descripcion_problema TEXT NOT NULL,  
  solucion TEXT NULL,  
  resultado ENUM('reparado', 'dado_de_baja', 'pendiente') DEFAULT 'pendiente',  
  fecha_solicitud TIMESTAMP NOT NULL,  
  fecha_completado TIMESTAMP NULL,  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL,  
  
  FOREIGN KEY (equipment_id) REFERENCES equipment(id) ON DELETE CASCADE,  
  FOREIGN KEY (requested_by) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (assigned_to) REFERENCES users(id) ON DELETE SET NULL,  
  INDEX idx_maintenance_status (status),  
  INDEX idx_maintenance_equipment_id (equipment_id),  
  INDEX idx_maintenance_assigned_to (assigned_to)  
);
```



YAMAGA
SYSTEMS

Tabla: audit_logs

```
CREATE TABLE audit_logs (  
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  event VARCHAR(255) NOT NULL,          -- 'loan_approved', 'loan_rejected', etc.  
  auditable_type VARCHAR(255) NOT NULL,  -- 'App\Models\Loan', 'App\Models\Equipment'  
  auditable_id BIGINT UNSIGNED NOT NULL,  
  user_id BIGINT UNSIGNED NOT NULL,      -- Quien ejecutó la acción  
  old_values JSON NULL,                  -- Estado anterior  
  new_values JSON NULL,                  -- Estado nuevo  
  description TEXT NULL,                 -- Descripción legible  
  ip_address VARCHAR(45) NULL,  
  user_agent VARCHAR(255) NULL,  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL,  
  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  INDEX idx_audit_auditable (auditable_type, auditable_id),  
  INDEX idx_audit_event (event),  
  INDEX idx_audit_user_id (user_id),  
  INDEX idx_audit_created_at (created_at)  
);
```

Tabla: system_settings

```
CREATE TABLE system_settings (  
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  key VARCHAR(255) UNIQUE NOT NULL,  
  value TEXT NULL,  
  type VARCHAR(255) DEFAULT 'string',   -- 'string', 'integer', 'boolean', 'json'  
  description VARCHAR(255) NULL,  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL,  
  
  INDEX idx_system_settings_key (key)  
);
```

Datos de Demostración

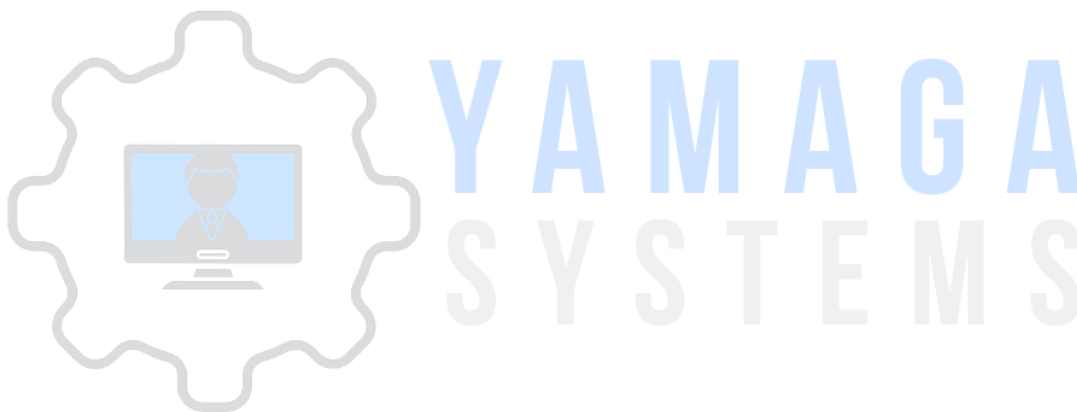
15 Usuarios creados:

- 3 Administradores (admin, laura, roberto)
- 8 Trabajadores (carlos, maria, juan, sofia, diego, valentina, lucas, camila)
- 4 Personal de Mantenimiento (pedro, ana, fernando, patricia)

41 Equipos creados:

- 10 Laptops (Dell, HP, Lenovo, MacBook)
- 8 Tablets (iPad, Samsung Galaxy Tab)
- 6 Proyectoros (Epson, BenQ)
- 5 Cámaras (Canon, Sony, Nikon)
- 5 Monitores (Dell, Samsung, LG)
- 4 Equipos de Audio (Micrófonos, Altavoces)
- 3 Equipos de Red (Routers, Switches)

Ver detalle completo en: [docs/DATOS_DEMO.md](#)



Manual de usuario

Introducción al Manual de Usuario

El sistema de Gestión de oficina cuenta con una interfaz web moderna y responsiva que funciona en cualquier dispositivo. A continuación se presenta una guía detallada de uso para cada rol de usuario, acompañada de capturas de pantalla que muestran las funcionalidades en acción.

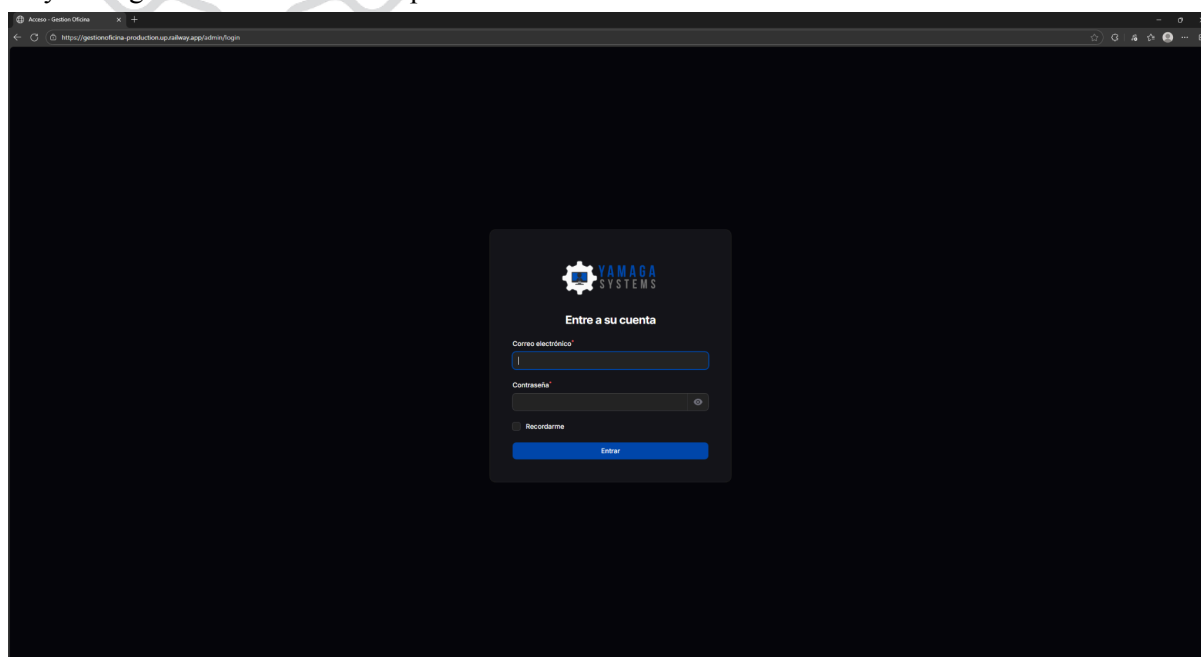
Acceso al Sistema:

- URL: <https://gestionoficina-production.up.railway.app/admin>
- Compatible con Chrome, Firefox, Edge, Safari
- Funciona en Desktop, Tablet y Móvil

Organización del Manual: El manual está dividido en tres secciones principales, una por cada rol de usuario. Cada sección incluye capturas de pantalla numeradas que corresponden a las instrucciones paso a paso.

Inicio de Sesión

Todos los usuarios acceden al sistema a través de la misma pantalla de login. El sistema identificará el rol y redirigirá al dashboard correspondiente.



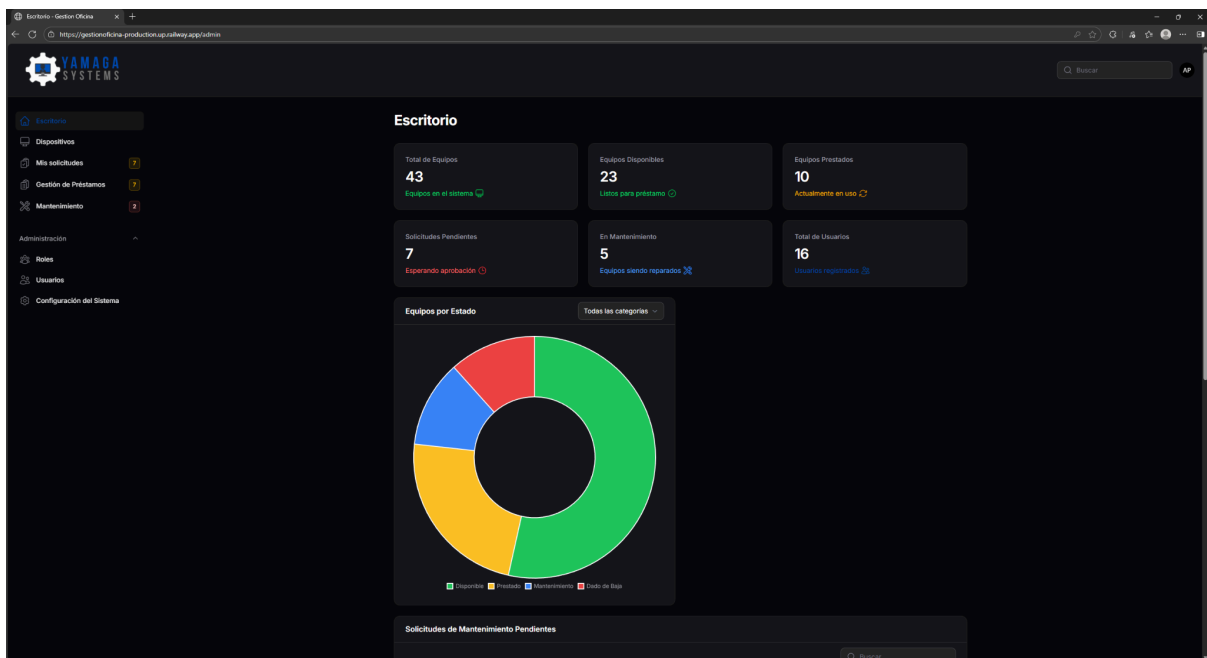
Pasos para iniciar sesión:

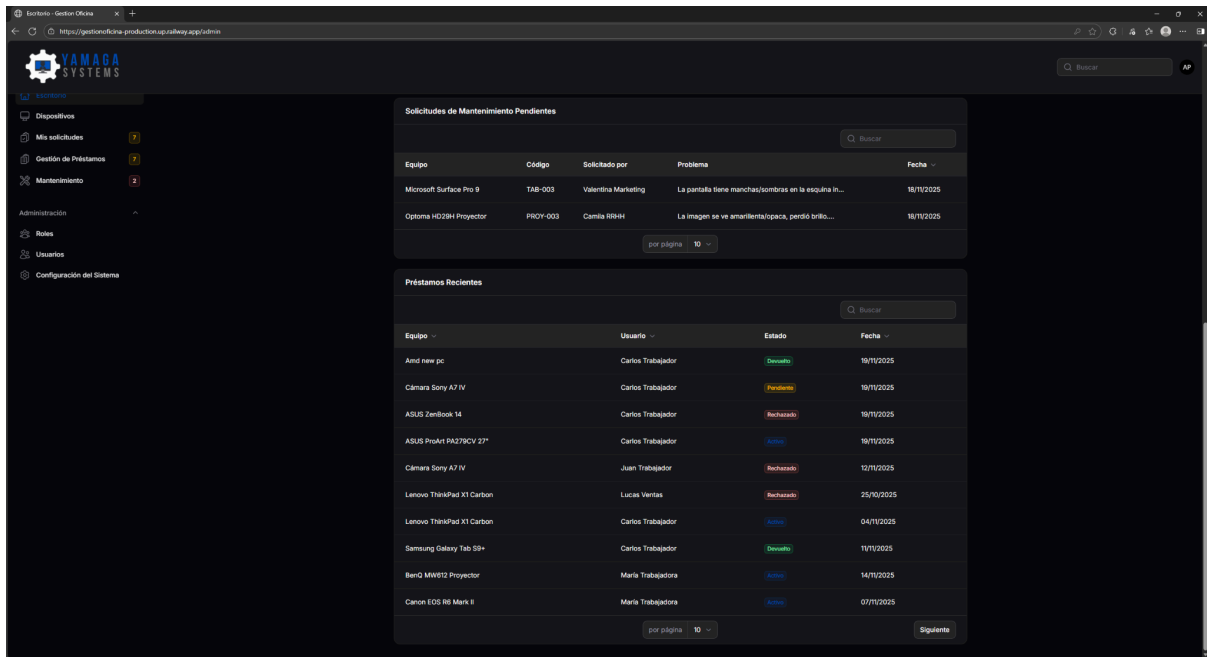
1. Abrir la URL del sistema en el navegador
2. Ingresar email y contraseña proporcionados por el administrador
3. Hacer click en “Iniciar Sesión”
4. El sistema redirige al dashboard según el rol

Manual del Administrador

Los administradores tienen control completo del sistema y acceso a todos los módulos.

Dashboard Principal del Administrador



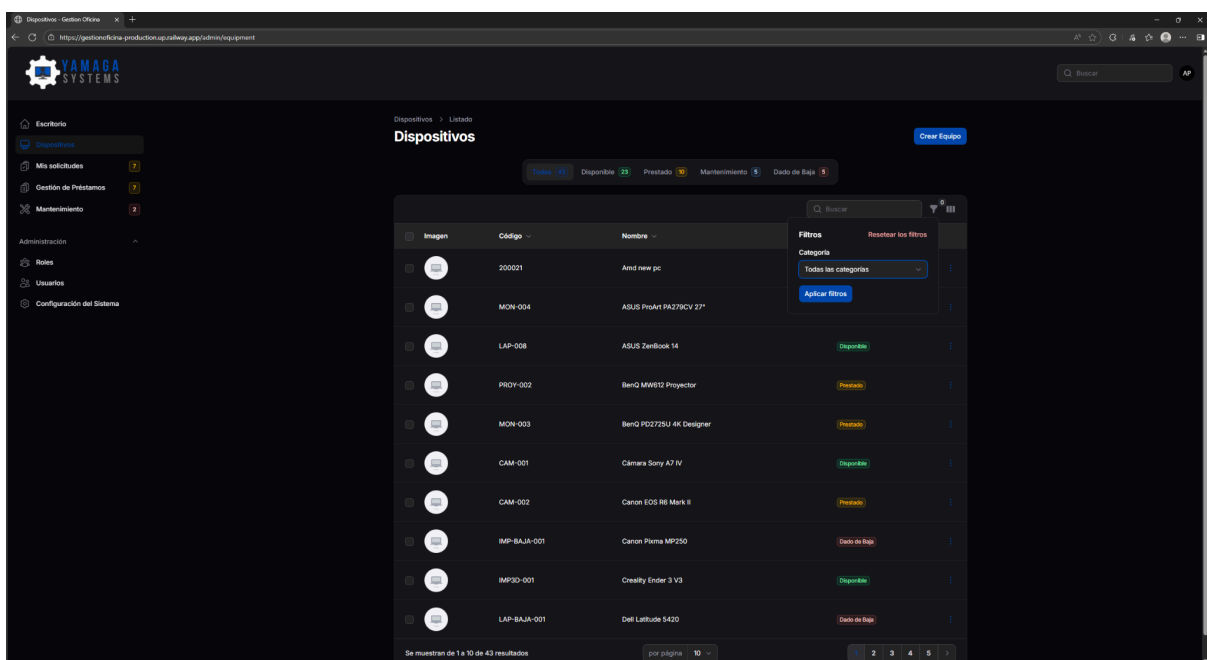


El dashboard incluye:

- Estadísticas generales (Total equipos, Prestados, Disponibles, En mantenimiento)
- Gráfico circular de distribución de equipos por estado
- Tabla de préstamos recientes
- Badges de notificación en el menú lateral

Gestión de Equipos

Ver y Filtrar Equipos



Características visibles:

- Barra de búsqueda por nombre
- Filtro por estado (disponible, prestado, mantenimiento, baja)
- Badges de color indicando estado de cada equipo
- Columnas ordenables
- Acciones disponibles por equipo

Crear Nuevo Equipo

The screenshot shows a web application interface for creating a new piece of equipment. The browser address bar shows the URL: `https://gestioneoficina-production.up.railway.app/admin/equipment/create`. The application has a dark theme and a sidebar on the left with navigation links: Escritorio, Dispositivos, Mis solicitudes, Gestión de Préstamos, Mantenimiento, Administración, Roles, Usuarios, and Configuración del Sistema. The main content area is titled 'Crear Equipo' and contains the following fields:

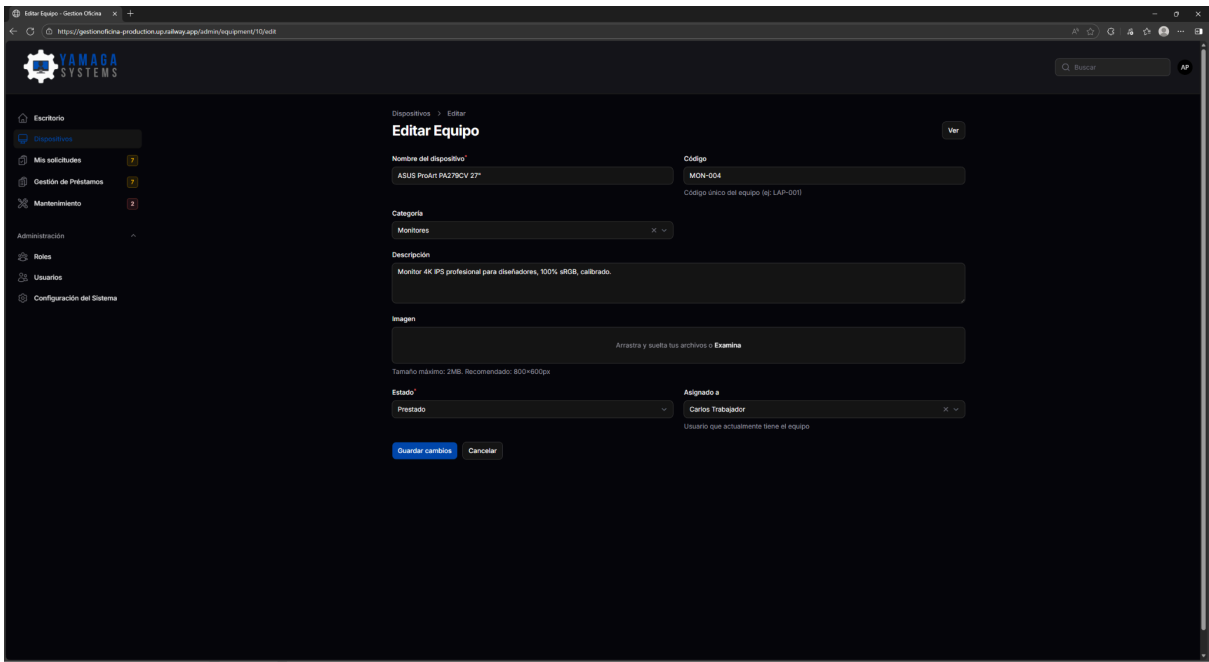
- Nombre del dispositivo***: A required text input field.
- Código**: A text input field with a placeholder 'Código único del equipo (ej. LAP-001)'.
- Categoría**: A dropdown menu with the option 'Selecciona una opción'.
- Descripción**: A large text area for a detailed description.
- Imagen**: A file upload area with the text 'Arrastra y suelta tus archivos o Examina'. Below it, a note states 'Tamaño máximo: 2MB. Recomendado: 800x600px'.
- Estado***: A required dropdown menu with 'Disponible' selected.
- Asignado a**: A dropdown menu with 'Selecciona una opción' and a note 'Usuario que actualmente tiene el equipo'.

At the bottom of the form are three buttons: 'Crear' (highlighted in blue), 'Crear y crear otro', and 'Cancelar'.

Campos del formulario:

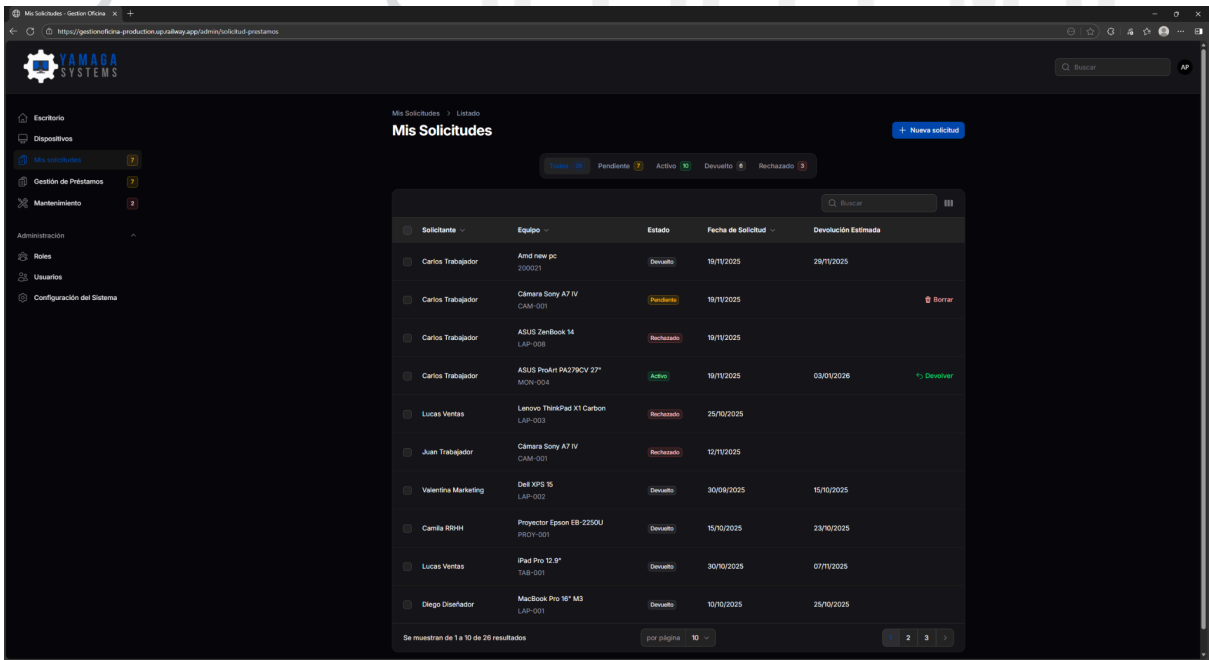
- Nombre (obligatorio)
- Descripción (opcional, texto largo)
- Estado inicial (selector con opciones)
- Imagen (carga de archivo, opcional)

Editar Equipo Existente



Muestra cómo los datos del equipo se cargan automáticamente para su modificación.

Ver Historial de Préstamos



Información visible:

- Usuario que tuvo el equipo
- Fechas de préstamo y devolución
- Estados
- Administrador que aprobó

Gestión de Solicitudes de Préstamo

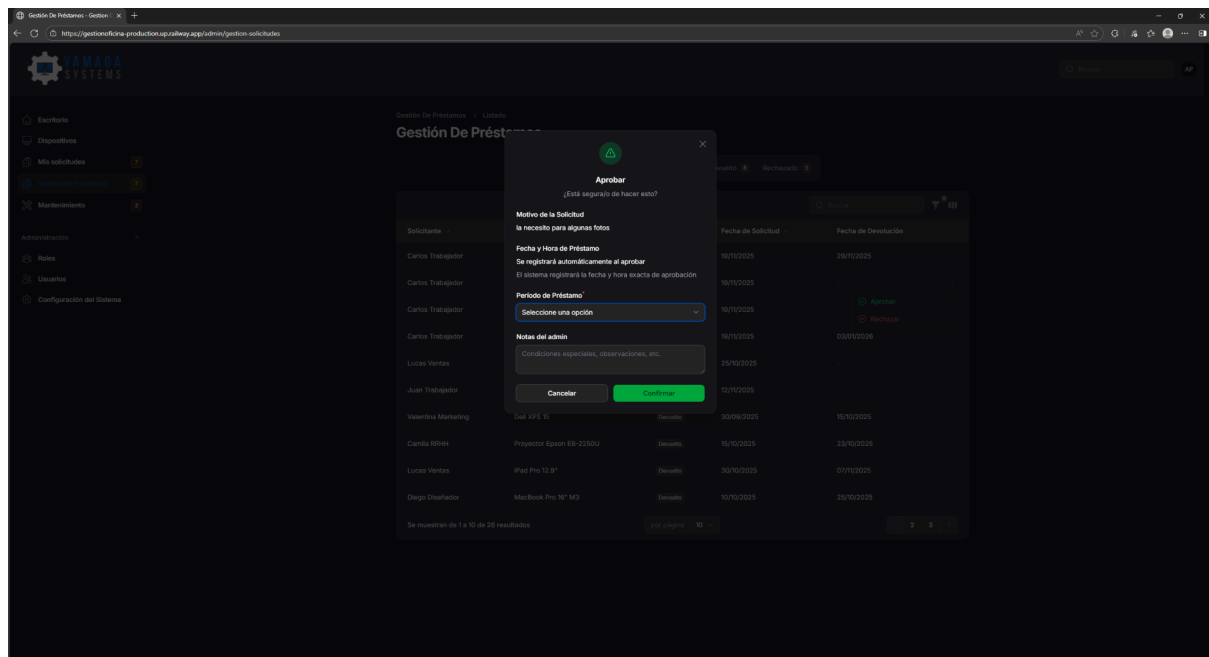
Listado de Solicitudes

Solicitante	Equipo	Estado	Fecha de Solicitud	Fecha de Devolución
Carlos Trabajador	Amd new pc	Devuelto	18/11/2025	28/11/2025
Carlos Trabajador	Cámara Sony A7 IV	Pendiente	18/11/2025	-
Carlos Trabajador	ASUS ZenBook 14	Rechazado	18/11/2025	-
Carlos Trabajador	ASUS ProArt PA378CV 27"	Activo	18/11/2025	03/01/2026
Lucas Ventas	Lenovo ThinkPad X1 Carbon	Rechazado	25/10/2025	-
Juan Trabajador	Cámara Sony A7 IV	Rechazado	12/11/2025	-
Valentina Marketing	Dell XPS 15	Devuelto	30/06/2025	15/10/2025
Camila Bidei	Projector Epson EB-2350U	Devuelto	15/10/2025	23/10/2025
Lucas Ventas	iPad Pro 12.9"	Devuelto	30/10/2025	07/11/2025
Diego Diseñador	MacBook Pro 16" M3	Devuelto	10/10/2025	25/10/2025

Elementos importantes:

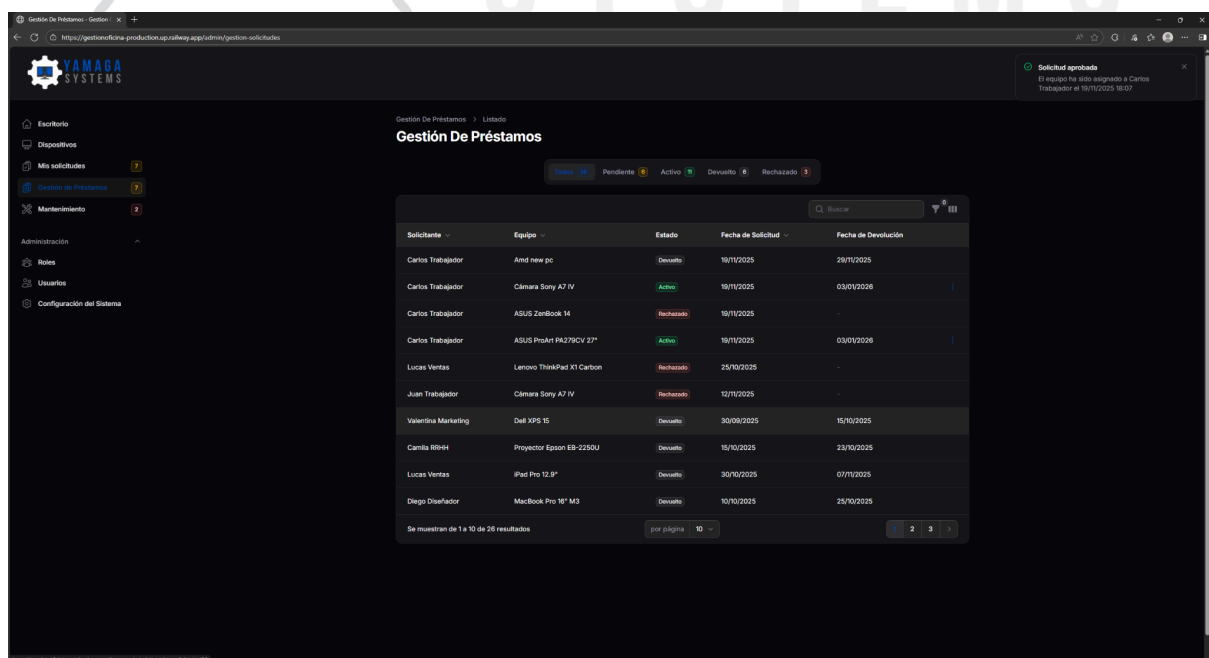
- Badge rojo/amarillo con número de pendientes
- Filtros por estado, usuario, equipo
- Badges de color por estado (amarillo=pendiente, verde=activo, rojo=rechazado, gris=devuelto)

Aprobar Solicitud



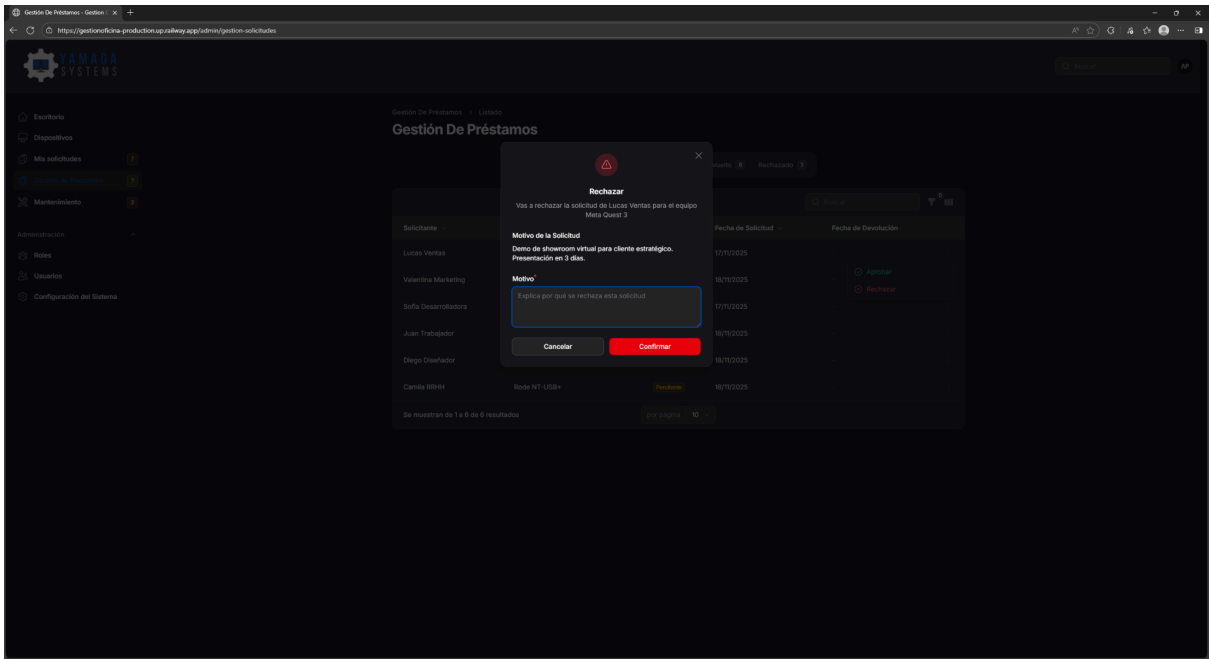
Campos visibles:

- Fecha de préstamo (auto-completada con fecha actual)
- Fecha de devolución (selector de calendario)
- Notas para el trabajador (opcional)

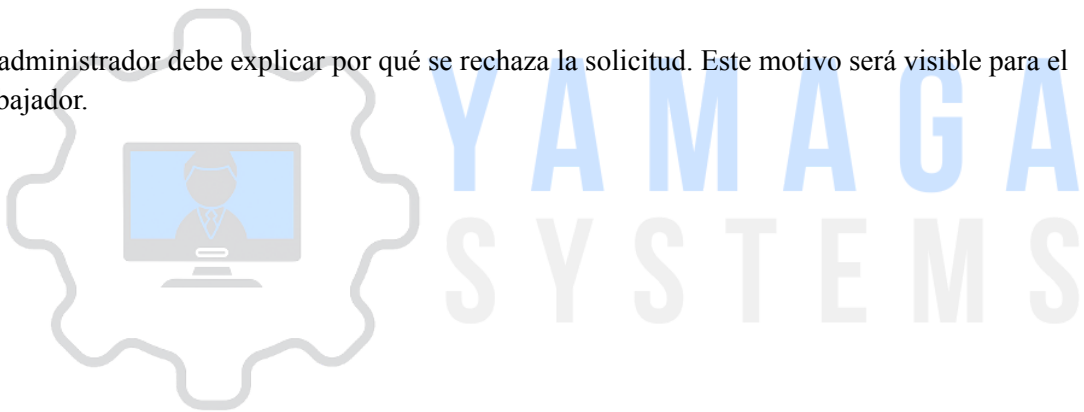


Muestra cómo el badge cambia de amarillo (pendiente) a verde (activo).

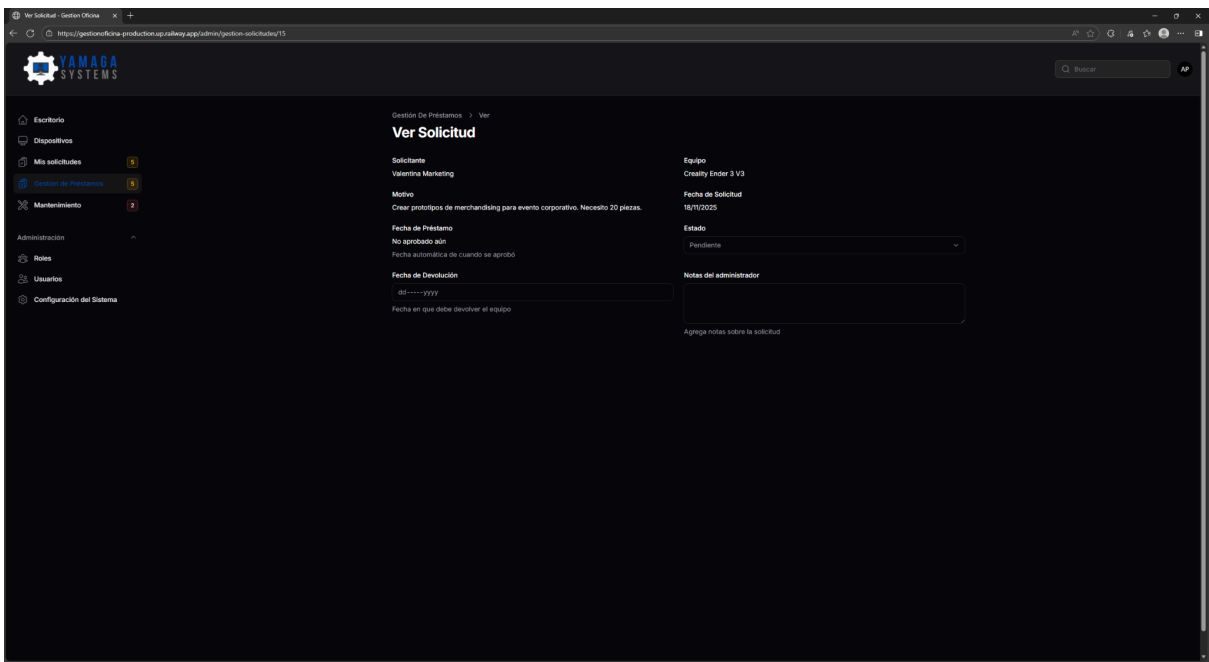
Rechazar Solicitud



El administrador debe explicar por qué se rechaza la solicitud. Este motivo será visible para el trabajador.



Ver Detalles de Solicitud

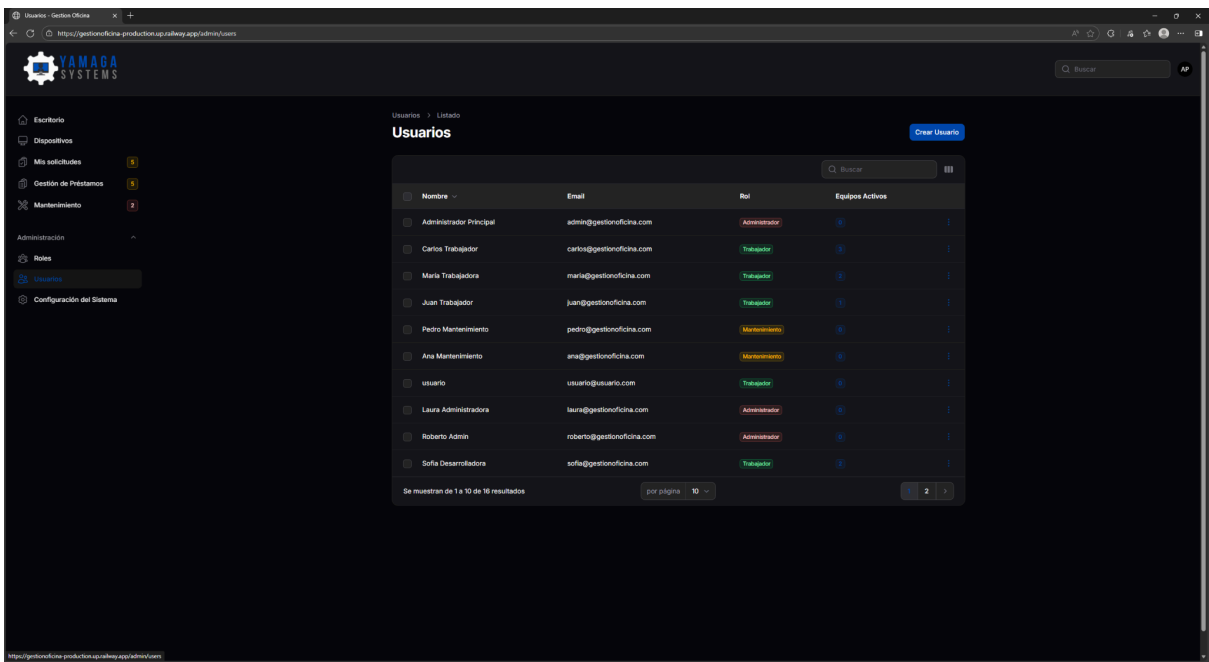


Información completa:

- Datos del equipo solicitado
- Información del trabajador
- Motivo de la solicitud
- Fechas relevantes
- Notas del administrador
- Estado actual

YAMAGA
SYSTEMS

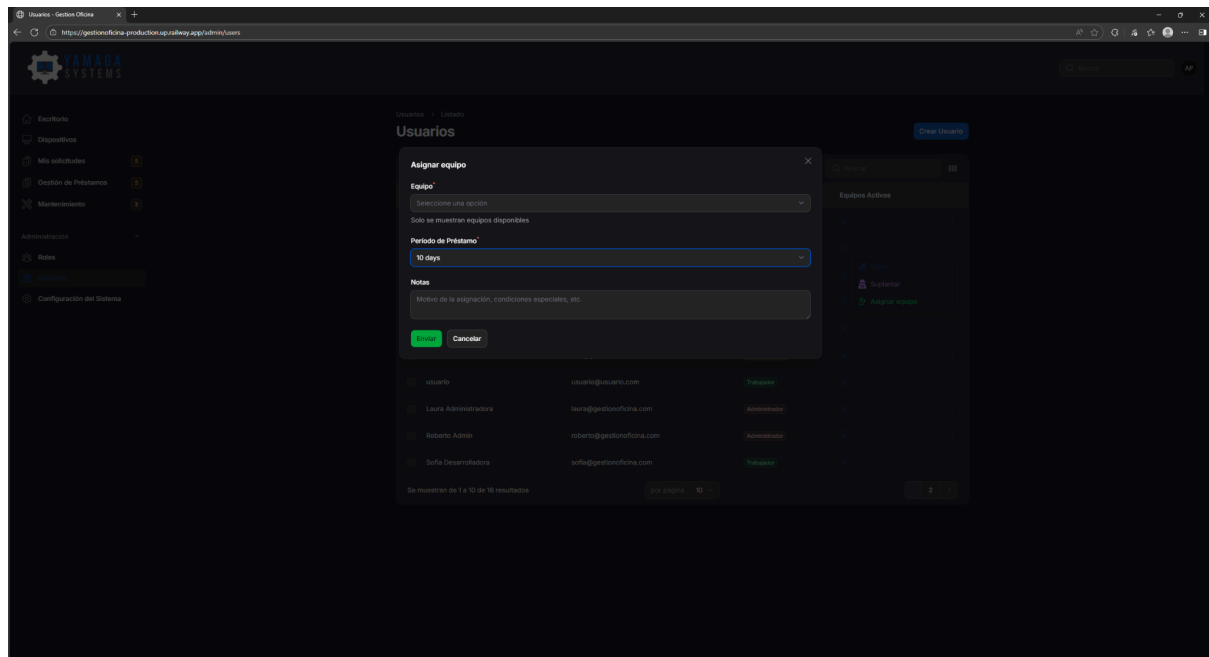
Gestion de Usuarios



Muestra:

- Nombre completo
- Email
- Rol (con badge de color: azul=Admin, verde=Trabajador, amarillo=Mantenimiento)
- Acciones disponibles

Asignar Equipo Directamente a Usuario



Esta funcionalidad permite al administrador asignar un equipo sin solicitud previa del trabajador. El modal incluye:

- Selector de trabajador (lista desplegable)
- Campo de fecha de devolución
- Campo de notas opcional

Crear Nuevo Usuario

Crear Usuario - Gestion Oficina

https://gestiofficina-production.up.railway.app/admin/usuarios/create

YAMAGA SYSTEMS

Buscar

AP

Escritorio

Dispositivos

Mis solicitudes 5

Gestión de Préstamos 5

Mantenimiento 2

Administración

Roles

Usuarios

Configuración del Sistema

Usuarios > Crear

Crear Usuario

Nombre*

Email*

Rol*

Seleccione una opción

Verificado al

dd-----yyyy --:--:--

Contraseña*

Mínimo 8 caracteres. Dejar en blanco para no cambiar.

Crear

Crear y crear otro

Cancelar

Crear Usuario - Gestion Oficina

https://gestiofficina-production.up.railway.app/admin/usuarios/create

YAMAGA SYSTEMS

Buscar

AP

Escritorio

Dispositivos

Mis solicitudes 5

Gestión de Préstamos 5

Mantenimiento 2

Administración

Roles

Usuarios

Configuración del Sistema

Usuarios > Crear

Crear Usuario

Nombre*

Julian

Email*

july5p@gmail.com

Rol*

Trabajador

Verificado al

19-Nov-2025 03:20:27 PM

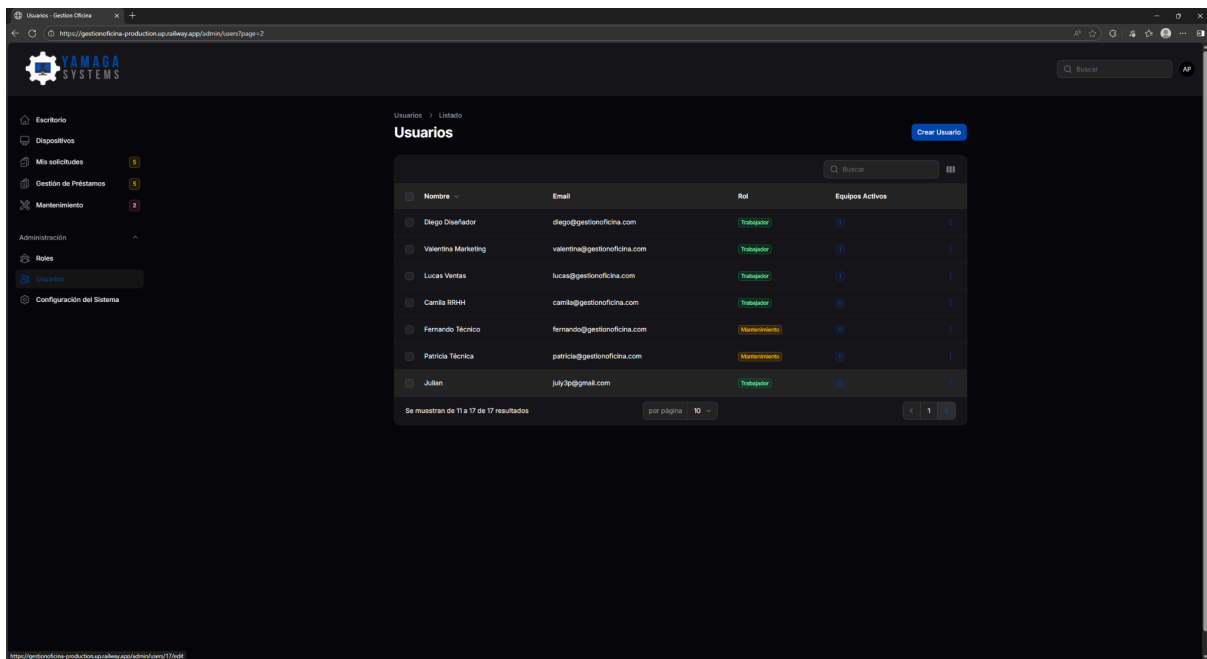
Contraseña*

Mínimo 8 caracteres. Dejar en blanco para no cambiar.

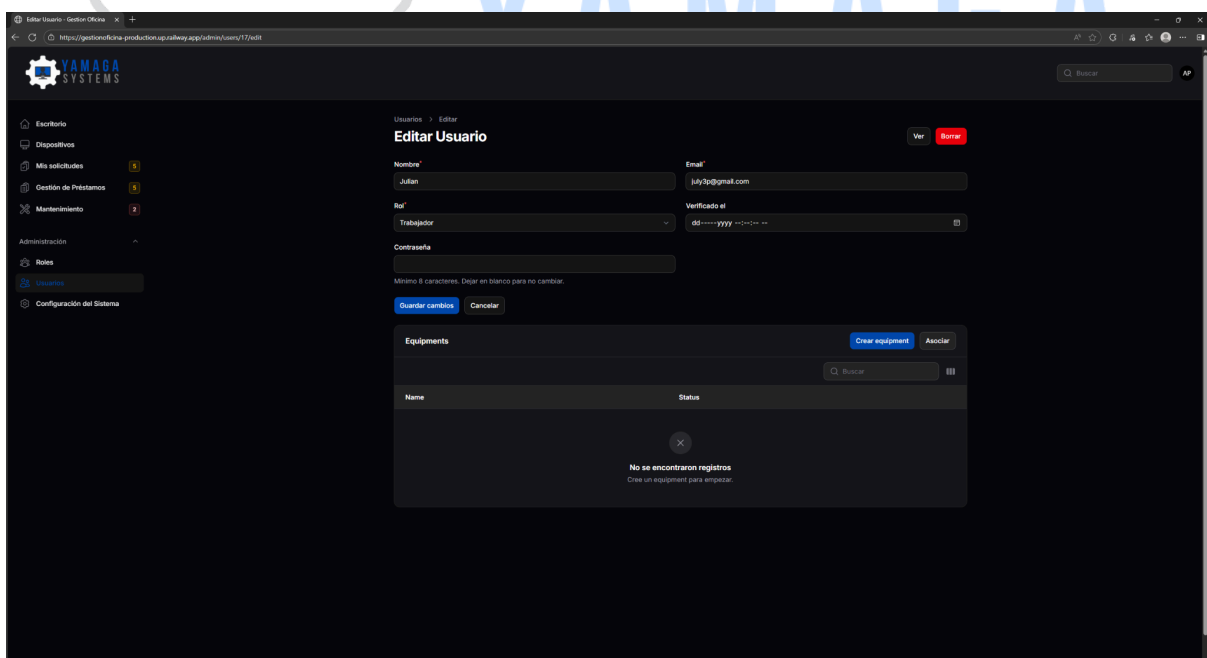
Crear

Crear y crear otro

Cancelar



Editar Usuario



Nota importante visible: El campo de contraseña es opcional al editar. Solo se llena si se desea cambiar la contraseña.

Gestión de Mantenimiento (Vista Admin)

Equipo	Estado	Problema
Amd new pc 200021	Completado	Se rompió la pantalla
Canon Pisma MP250 IMP-BAJA-001	Completado	No imprime colores. Solo sale negro con...
Samsung SyncMaster 2243 MON-BAJA-001	Completado	No enciende. Huíle a quemado. LED de pow...
Epson PowerLite 2005 PROY-BAJA-001	Completado	La lámpara fundida, colores distorsionad...
iPad Air 2019 TAB-BAJA-001	Completado	Se cayó y la pantalla se rompió completa...
Dell Latitude 5420 LAP-BAJA-001	Completado	Se apagó de repente y no enciende más. N...
Synology DS923+ NAS ALM-002	En Proceso	Disco 3 muestra advertencia SMART. Tempe...
Ubiquiti UniFi Dream Machine NET-003	En Proceso	WiFi se desconecta cada 30 minutos. Put...
Optoma HD29H Projector PROY-003	Pendiente	La imagen se ve amarillenta/opaca, pend...
Microsoft Surface Pro 9 TAB-003	Pendiente	La pantalla tiene manchas/sombras en la...

El administrador ve todas las solicitudes, no solo las asignadas a él. Incluye:

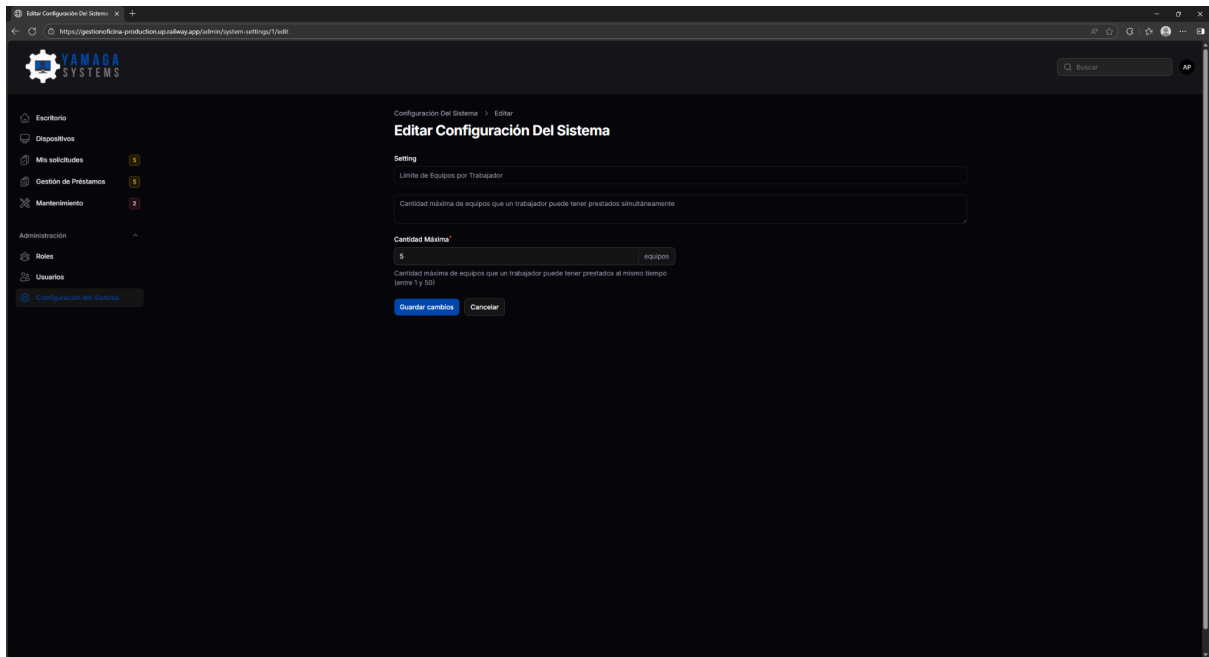
- Equipo en mantenimiento
- Reportado por
- Descripción del problema
- Técnico asignado
- Estado y resultado

Configuración del Sistema

Clave	Valor	Tipo	Descripción	Última actualización
days_before_return_warning	7	Número	Días antes de la fecha de devolución para mostrar...	16/11/2025 16:26
max_equipment_per_worker	5	Número	Cantidad máxima de equipos que un trabajador puede...	16/11/2025 16:26

Parámetros visibles:

- max equipments_per_worker (valor actual: 5)
- dias_aviso_vencimiento (valor actual: 7)

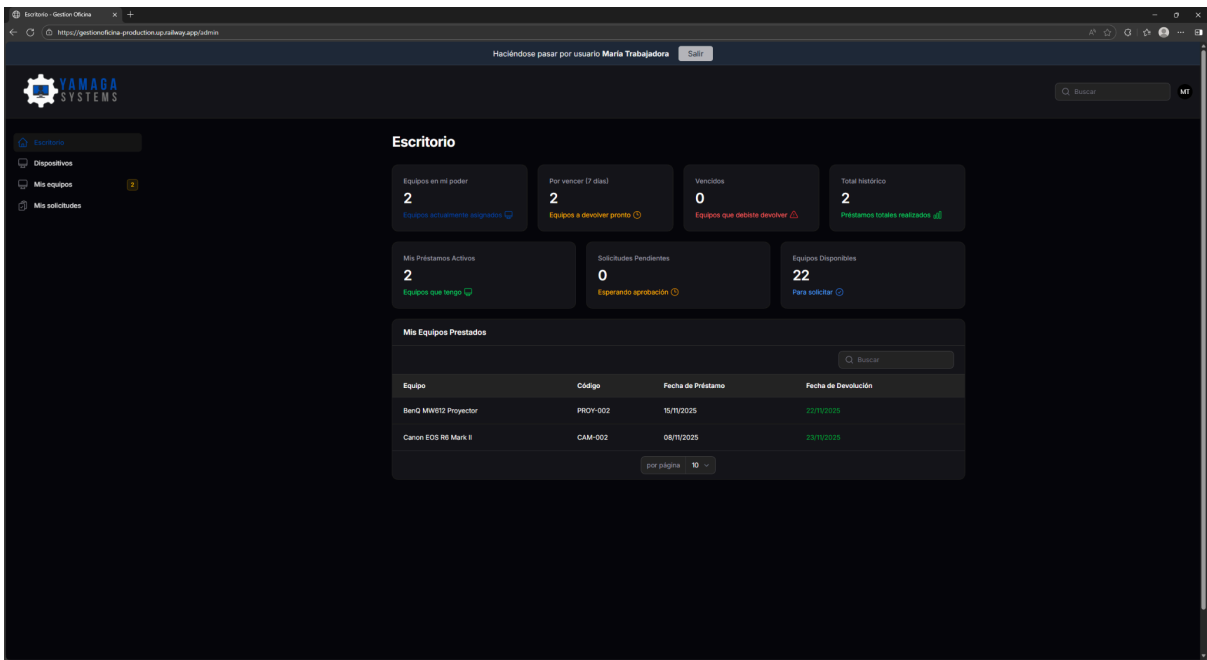


Muestra cómo cambiar el valor de un parámetro. Los cambios se aplican inmediatamente.

Manual del Trabajador

Los trabajadores tienen acceso limitado a funcionalidades relacionadas con sus propios préstamos y equipos.

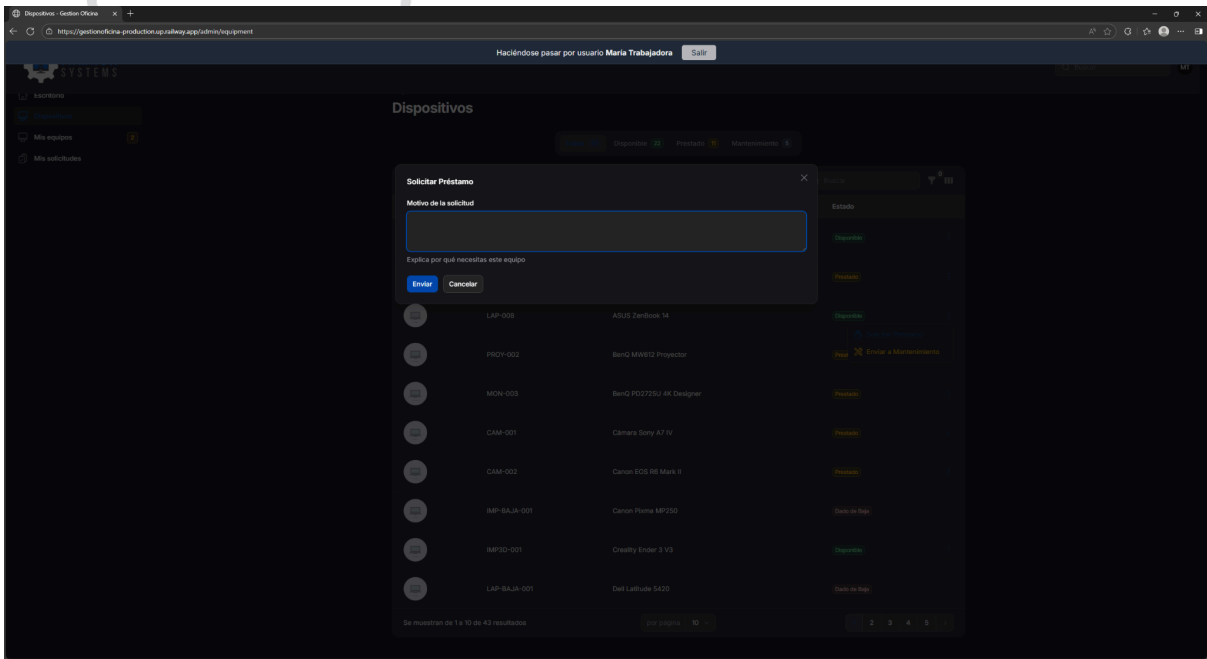
Dashboard del Trabajador



Widgets visibles:

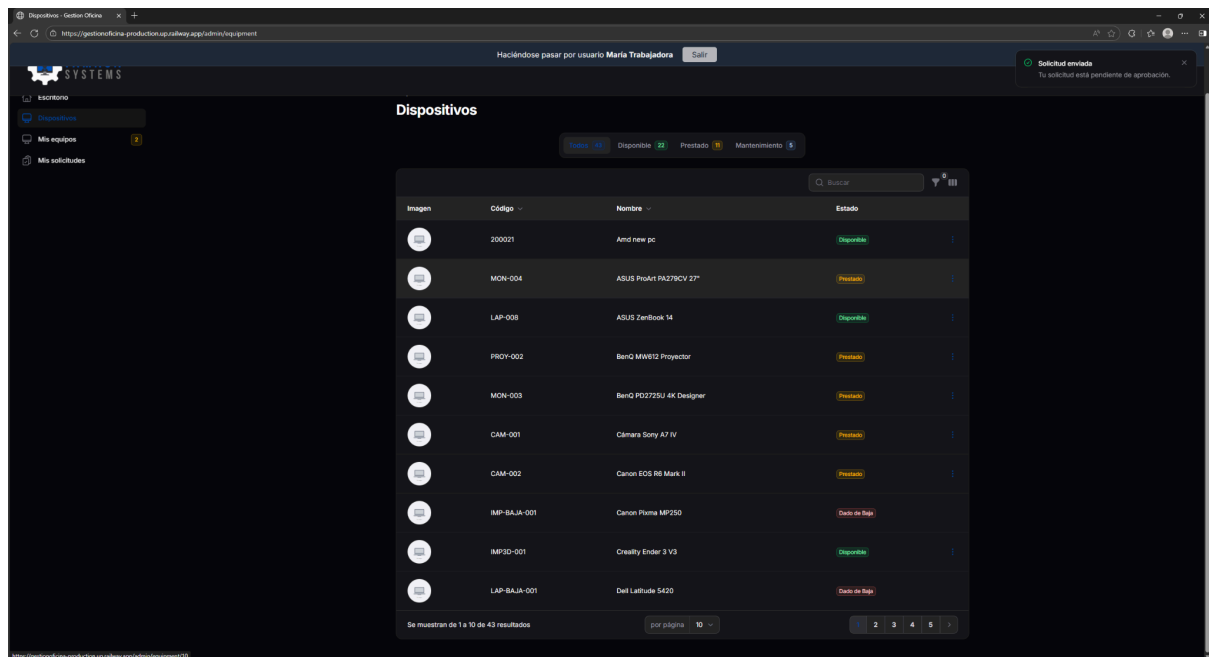
- "Mis Préstamos Activos" con equipos actuales
- "Mis Estadísticas" con números personales
- Alertas de vencimiento (si aplica)

Solicitar un Préstamo

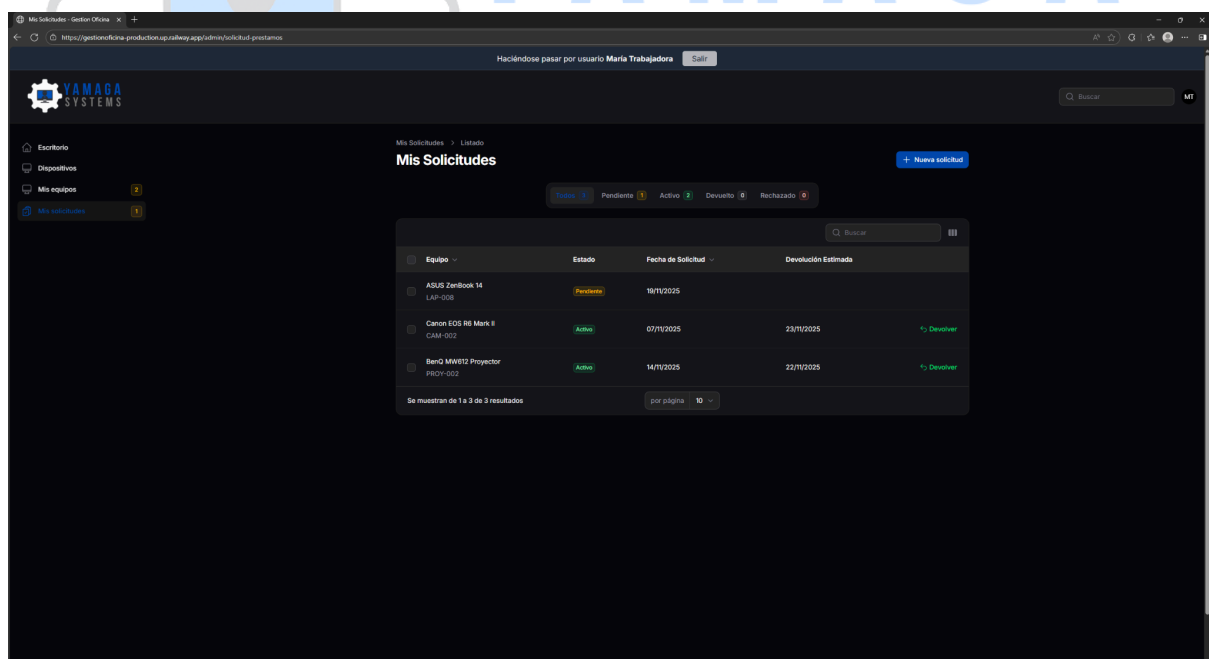


Campos:

- Equipo: Lista desplegable (solo muestra equipos disponibles)
- Motivo: Campo de texto obligatorio

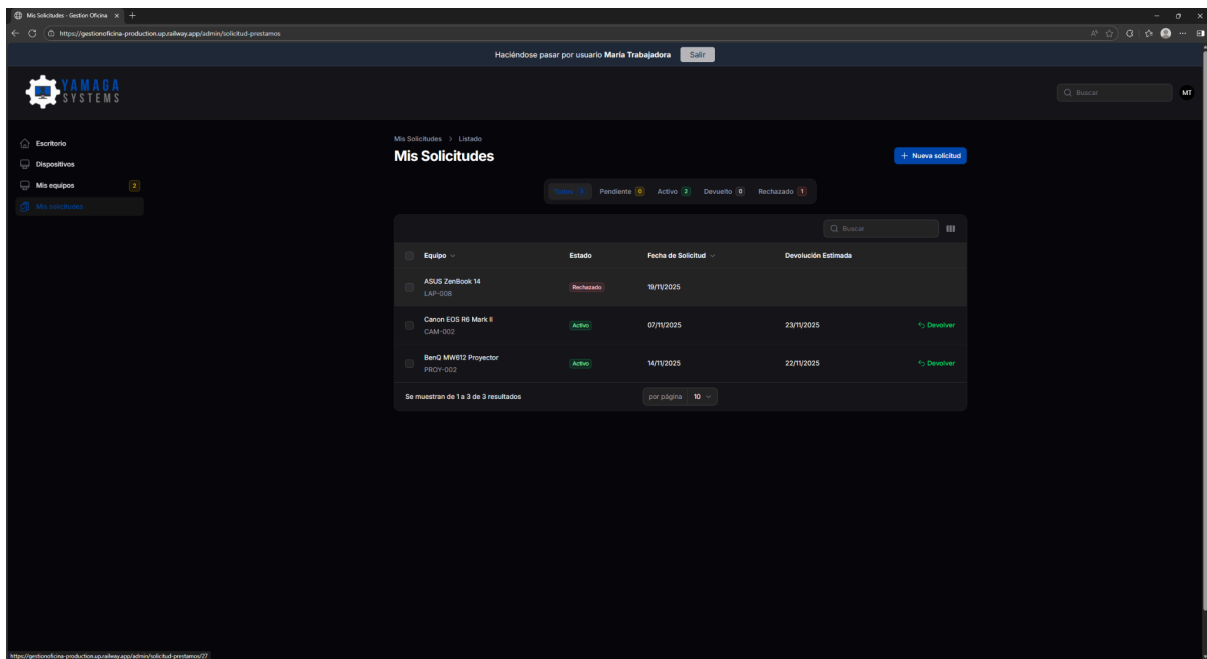


Mis solicitudes



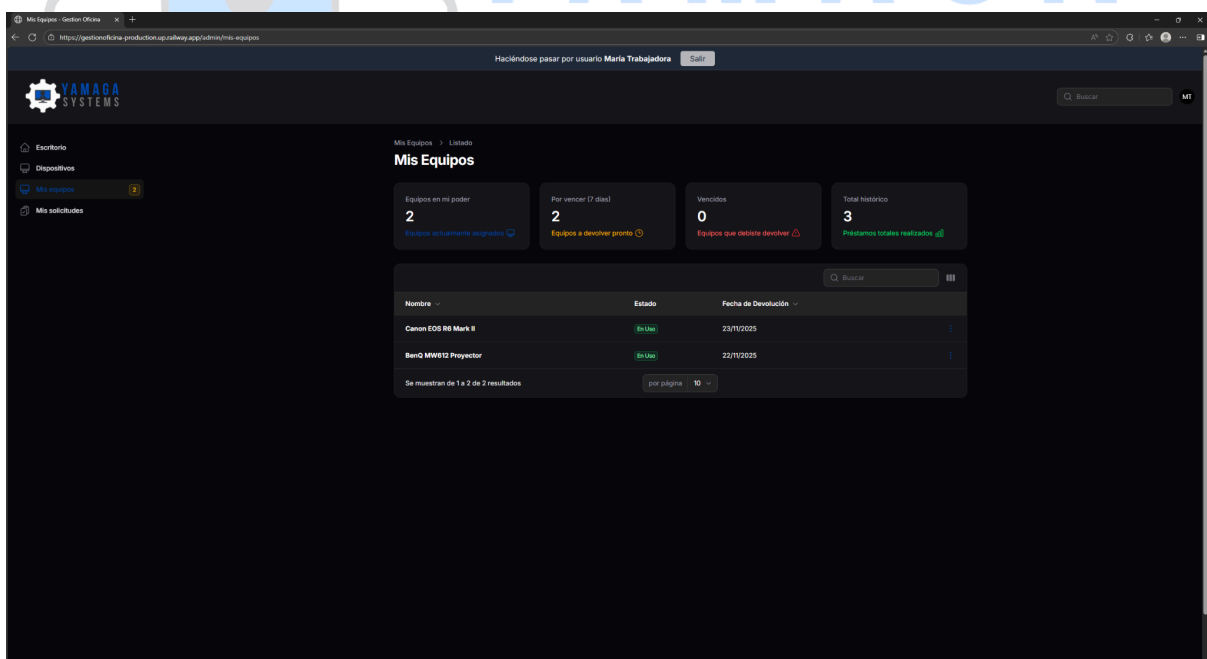
Estados visibles con badges de color:

- Pendiente (amarillo): Esperando aprobación
- Activo (verde): Aprobado, equipo en su poder
- Rechazado (rojo): Solicitud denegada
- Devuelto (gris): Préstamo finalizado



El trabajador puede ver si se rechazó su petición

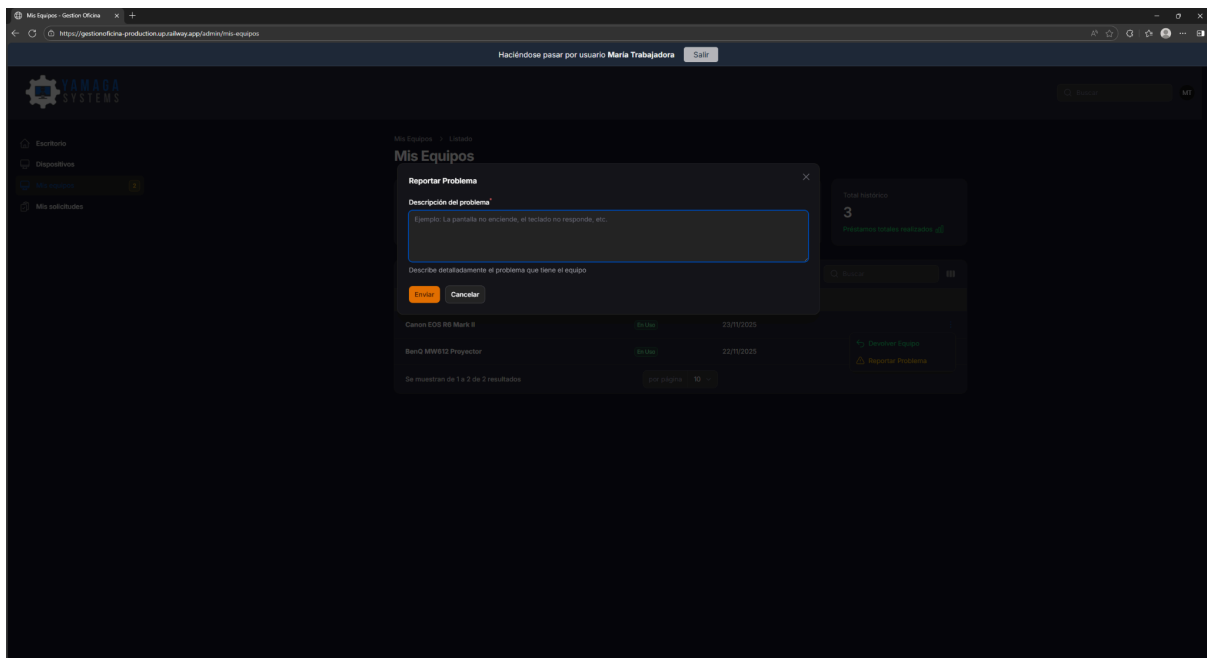
Mis equipos



Información mostrada:

- Nombre del equipo
- Fecha de préstamo
- Fecha de devolución estimada
- Alerta de vencimiento (si faltan 7 días o menos)

Reportar Problema



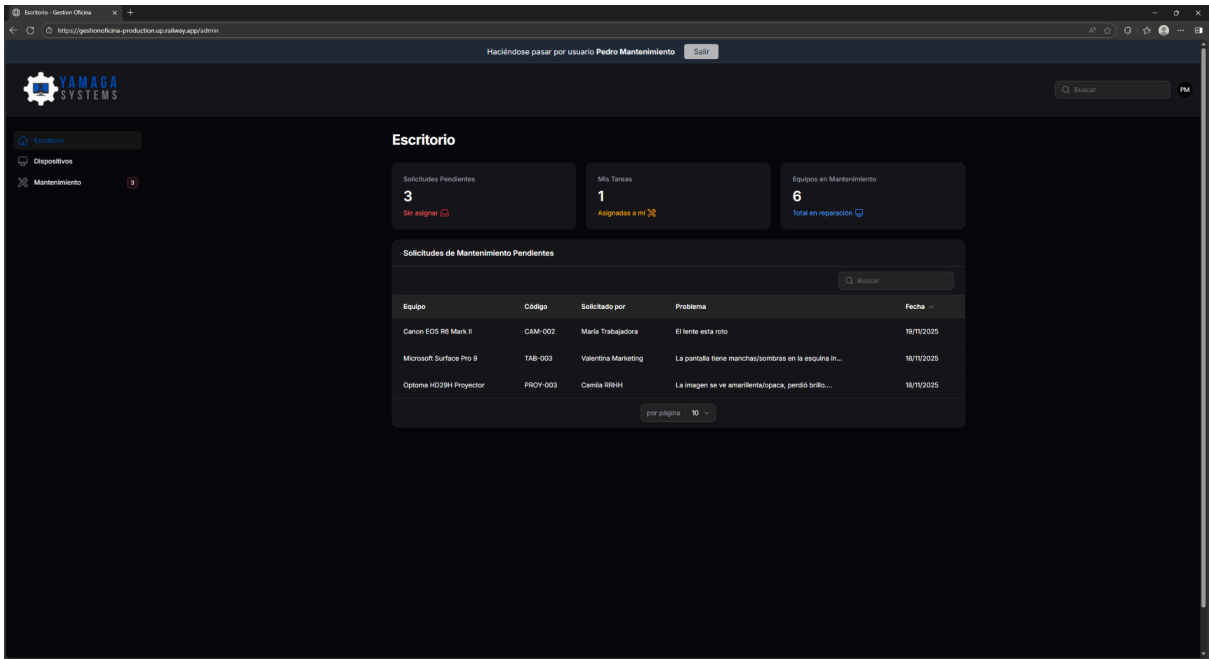
El trabajador puede reportar problemas técnicos en equipos asignados. El modal incluye:

- Campo de descripción del problema (obligatorio)
- Al reportar, el equipo pasa automáticamente a mantenimiento

Manual del Personal de Mantenimiento

El personal de mantenimiento gestiona las reparaciones de equipos reportados

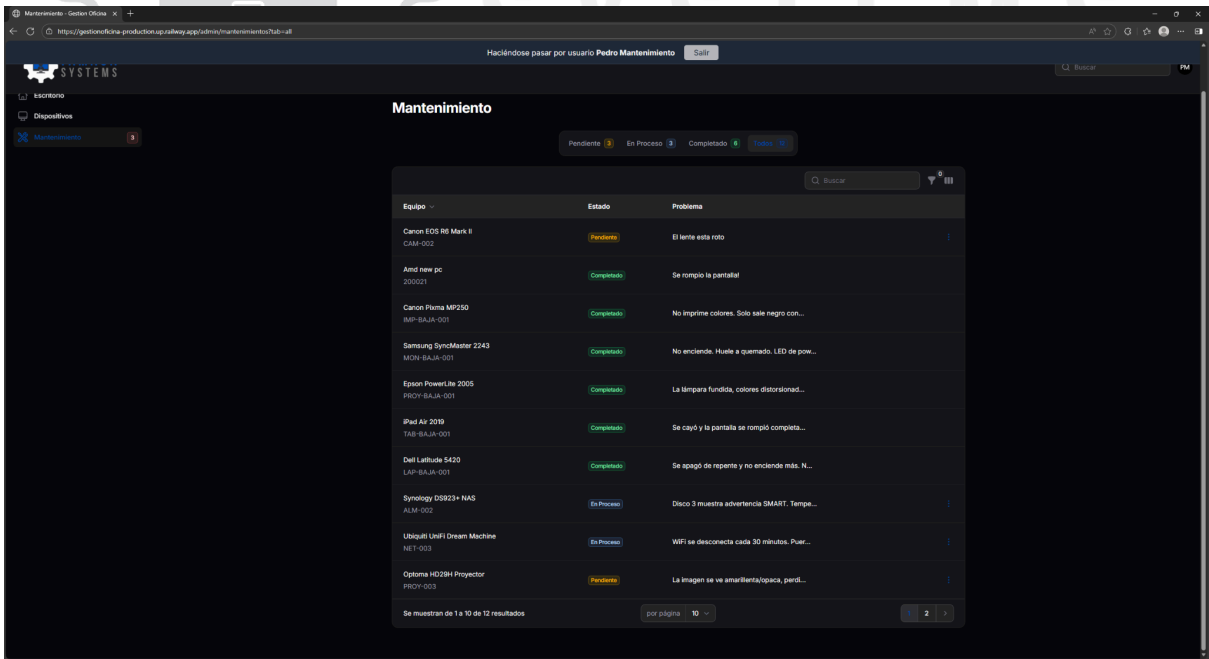
Dashboard de Mantenimiento



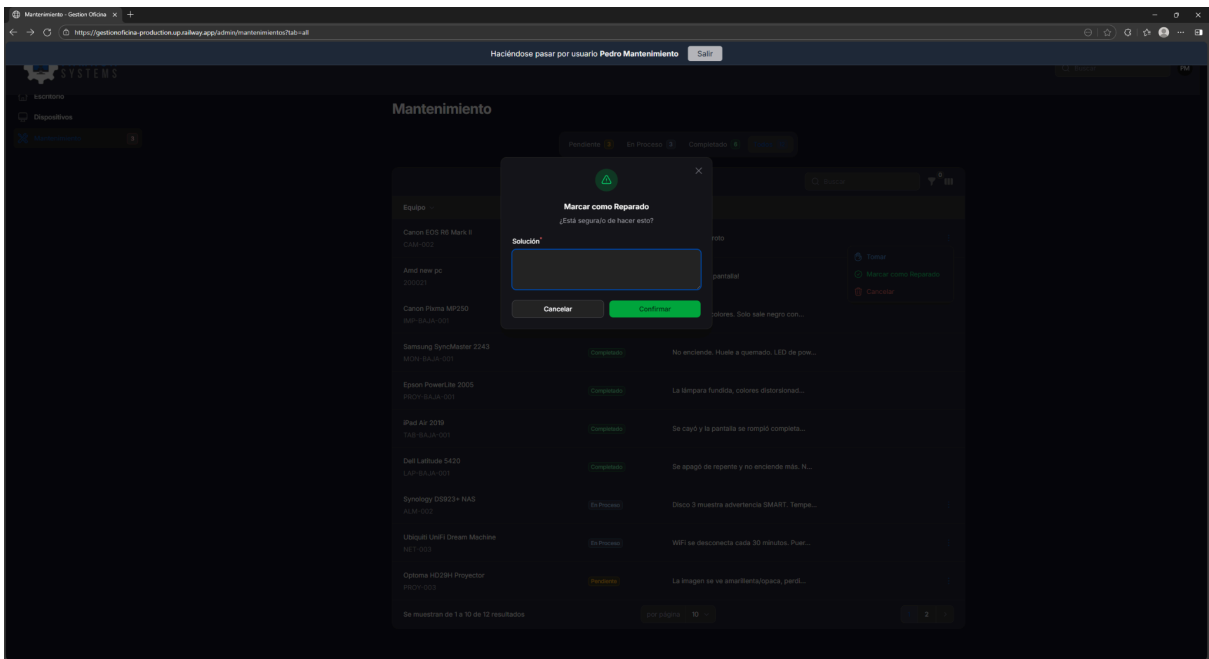
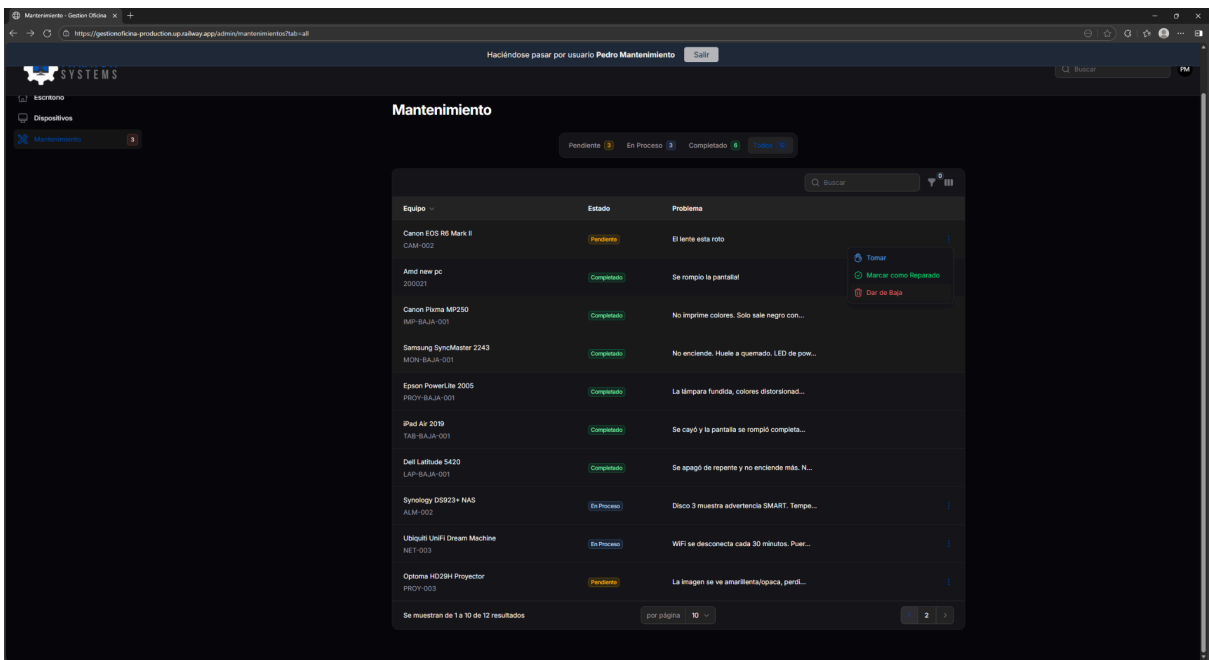
Widget "Solicitudes Pendientes":

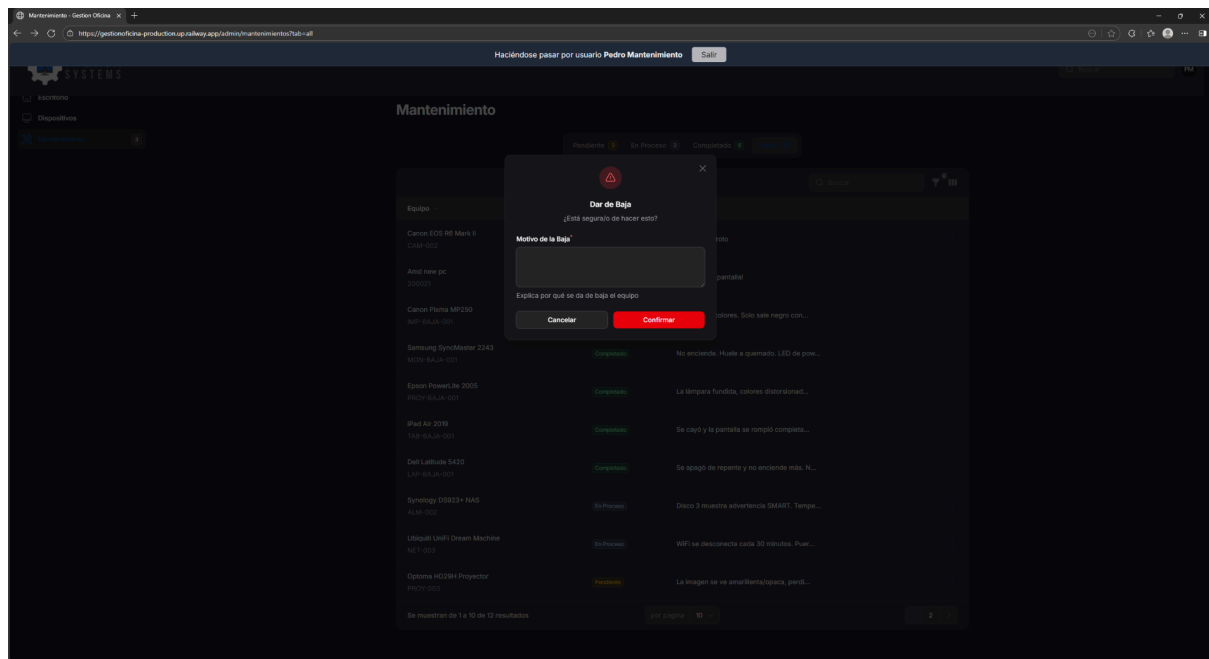
- Lista de equipos reportados
- Badge en menú con cantidad pendiente

Gestión de Solicitudes



Completar Mantenimiento





Campos:

- Solución aplicada (texto descriptivo)
- Resultado: "Reparado" o "Dado de Baja"

Ejemplo de solución: "Reemplazo de pantalla LCD modelo LP156WF6. Cable flex reinstalado correctamente. Testeo realizado durante 30 minutos sin problemas. Equipo funcionando correctamente."

Al marcar como reparado, el equipo vuelve automáticamente al estado "Disponible". Al dar de baja, el equipo pasa al estado "Baja" permanentemente.

Referencias

Documentacion Oficial

- Laravel 12: <https://laravel.com/docs/12.x>
- Filament 4: <https://filamentphp.com/docs/4.x/>
- MySQL 8.0: <https://dev.mysql.com/doc/refman/8.0/>
- Tailwind CSS: <https://tailwindcss.com/docs>
- Docker: <https://docs.docker.com/>

Repositorios

- Proyecto: <https://github.com/colomyago/gestionoficina>
- Laravel Sail: <https://github.com/laravel/sail>
- Filament: <https://github.com/filamentphp/filament>

Servicios Externos

- Railway: <https://railway.app>
- GitHub: <https://github.com>

Glosario

CRUD: Create, Read, Update, Delete - Operaciones básicas de base de datos

Eloquent: ORM (Object-Relational Mapping) de Laravel para interactuar con la base de datos

Middleware: Capa de software que filtra peticiones HTTP antes de llegar al controlador

Migration: Archivo que define cambios en el esquema de base de datos

Seeder: Archivo que inserta datos de prueba en la base de datos

Policy: Clase que define reglas de autorización para un modelo

Livewire: Framework de Laravel para componentes reactivos sin JavaScript

Filament: Framework de administración construido sobre Laravel y Livewire

Railway: Plataforma de deployment (PaaS - Platform as a Service)

Docker: Plataforma de contenerización para desarrollo

Sail: Wrapper de Docker específico de Laravel

Widget: Componente visual del dashboard (estadísticas, gráficos, etc.)

Badge: Indicador visual numérico (ej: contador de notificaciones)

Eager Loading: Técnica de optimización que carga relaciones en una sola query

Soft Delete: Borrado lógico (marca como eliminado sin borrar físicamente)

Foreign Key: Clave foránea que relaciona dos tablas

Index: Estructura de datos que mejora velocidad de búsqueda en BD

Transaction: Conjunto de operaciones de BD que se ejecutan atómicamente

Auditable: Modelo que registra automáticamente cambios en `audit_logs`

Enum: Tipo de dato con valores predefinidos (ej: estado = 'pendiente' | 'activo')